# UNIT – 2 and UNIT – 5 (Combined)

## What is Relational Model? - Tables

**RELATIONAL MODEL (RM)** represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

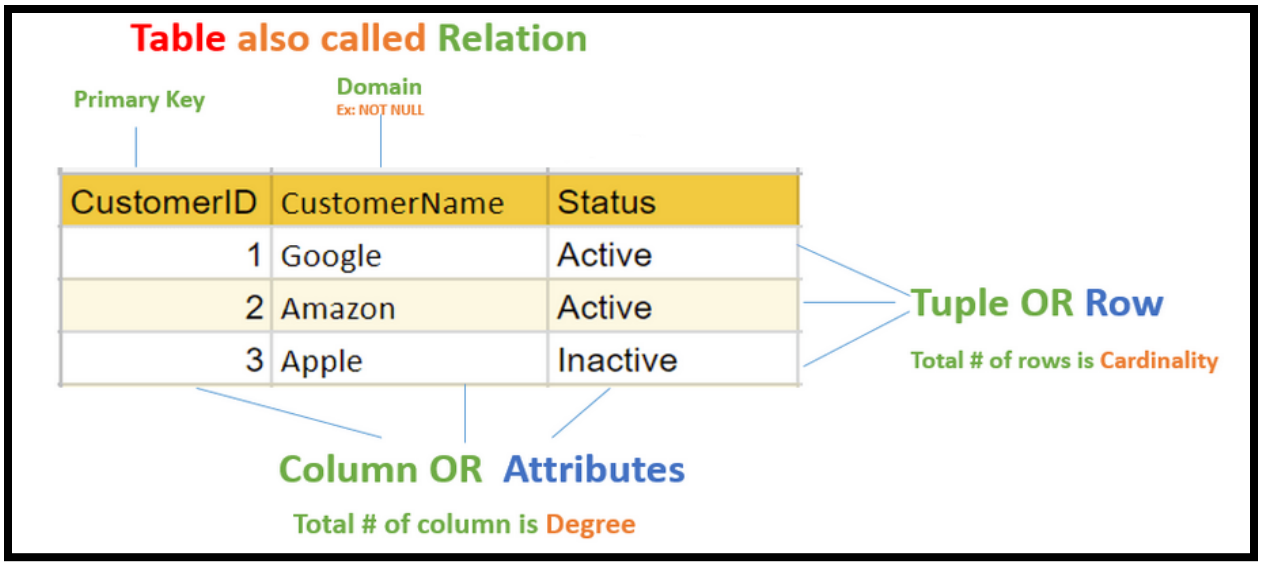
The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables. However, the physical storage of the data is independent of the way the data are logically organized.

Some popular Relational Database management systems are:

- DB2 and Informix Dynamic Server - IBM
- Oracle and RDB – Oracle
- SQL Server and Access - Microsoft

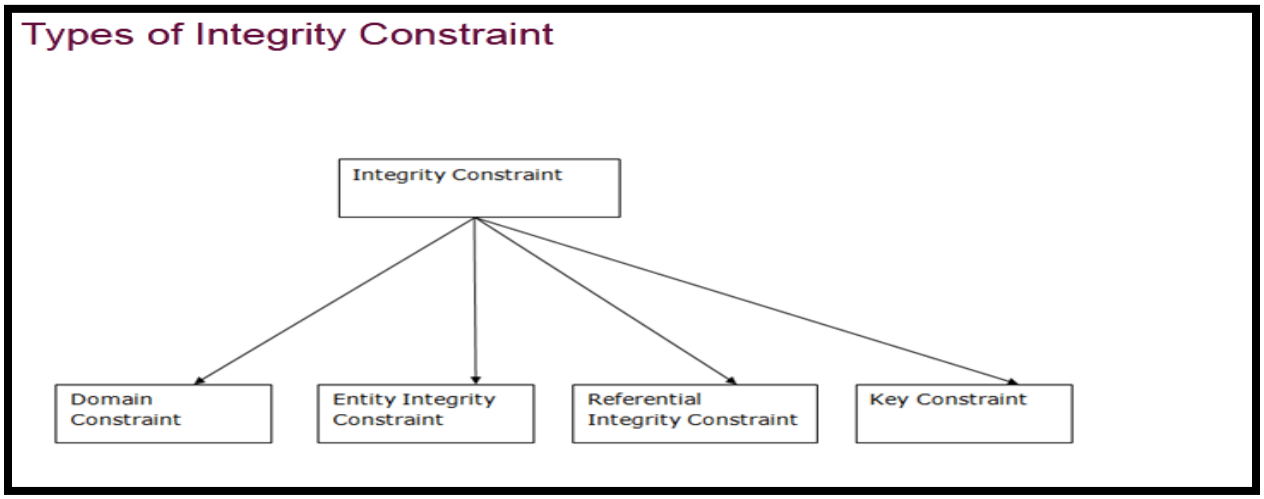
## Relational Model Concepts

1. **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME,etc.
2. **Tables** – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.
3. **Tuple** – It is nothing but a single row of a table, which contains a single record.
4. **Relation Schema:** A relation schema represents the name of the relation with its attributes. **Student(rollno, name, dept, marks)**
5. **Degree:** The total number of attributes present in the relation is called the degree of the relation.
6. **Cardinality:** Total number of rows present in the Table.
7. **Column:** The column represents the set of values for a specific attribute.
8. **Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
9. **Relation key** - Every row has one, two or multiple attributes, which is called relation key.
10. **Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain .




# Integrity Constraints (Rules)

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.




## 1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.

- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

**Example**

| ID | NAME | SEMENSTER | AGE |
|---|---|---|---|
| 1000 | Tom | 1st | 17 |
| 1001 | Johnson | 2nd | 24 |
| 1002 | Leonardo | 5th | 21 |
| 1003 | Kate | 3rd | 19 |
| 1004 | Morgan | 8th | A |

Not allowed. Because AGE is an integer attribute

## 2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

**Example:**

**EMPLOYEE**

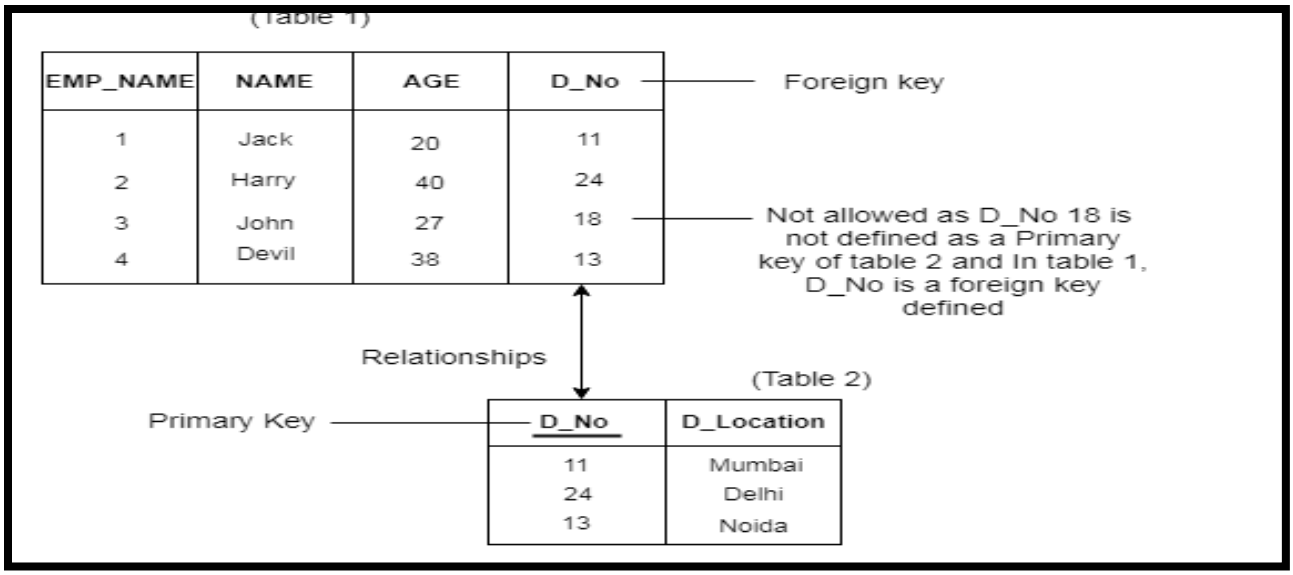
| EMP_ID | EMP_NAME | SALARY |
|---|---|---|
| 123 | Jack | 30000 |
| 142 | Harry | 60000 |
| 164 | John | 20000 |
| | Jackson | 27000 |

Not allowed as primary key can't contain a NULL value

## 3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

**Example**




## 4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

**Example**

| ID | NAME | SEMENSTER | AGE |
|---|---|---|---|
| 1000 | Tom | 1st | 17 |
| 1001 | Johnson | 2nd | 24 |
| 1002 | Leonardo | 5th | 21 |
| 1003 | Kate | 3rd | 19 |
| 1002 | Morgan | 8th | 22 |

Not allowed. Because all row must be unique

# Operations in Relational Model

Four basic update operations performed on relational database model are

Insert, update, delete and select.

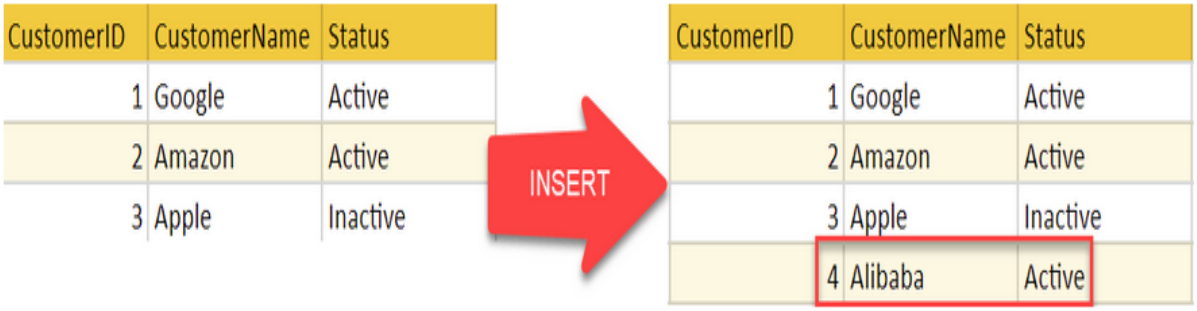
- Insert is used to insert data into the relation

- Delete is used to delete tuples from the table.
- Modify allows you to change the values of some attributes in existing tuples.
- Select allows you to choose a specific range of data.

Whenever one of these operations are applied, integrity constraints specified on the relational database schema must never be violated.

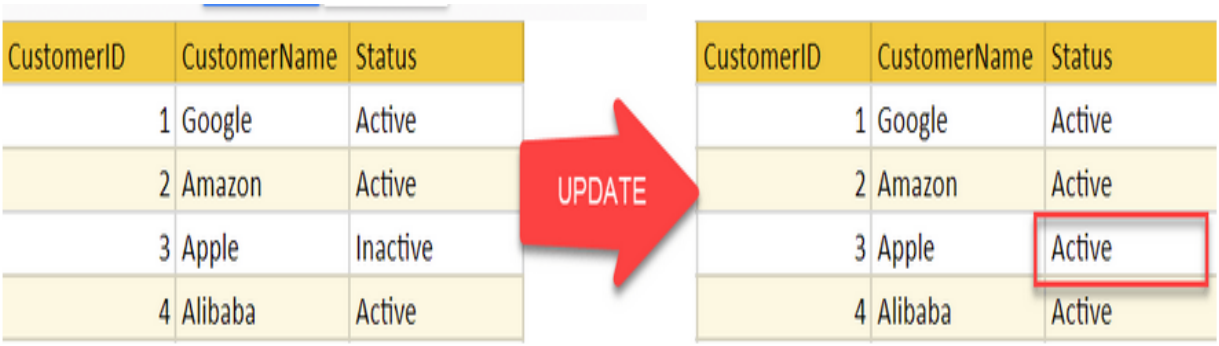
## Insert Operation

The insert operation gives values of the attribute for a new tuple which should be inserted into a relation.




## Update Operation

You can see that in the below-given relation table CustomerName= 'Apple' is updated from Inactive to Active.




## Delete Operation

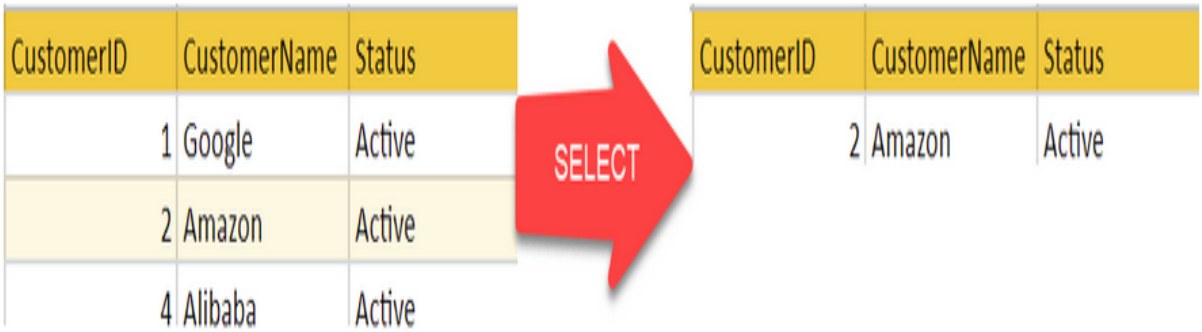
To specify deletion, a condition on the attributes of the relation selects the tuple to be deleted.




The Delete operation could violate referential integrity if the tuple which is deleted is referenced by foreign keys from other tuples in the same database.

## Select Operation

Fetch Data from the Data Base




In the above-given example, CustomerName="Amazon" is selected

## ORACLE HISTORY

| VERSION | DATE | RELEASE NAME | |
|---|---|---|---|
| | | ORACLE 6 | DBMS |
| | | ORACLE 7 | RDBMS |
| 8.0 | JUN 1997 | ORACLE 8 | JVM INCLUDED |
| 8.1.5 | FEB 1999 | ORACLE 8i Release 1 | I=Internet Based |
| 8.1.6 | NOV 1999 | ORACLE 8i Release 2 | |
| 8.1.7 | AUG 2000 | ORACLE 8i Release 3 | |
| 9.0.1 | JUN 2001 | ORACLE 9i Release 1 | |
| 9.2 | MAY 2002 | ORACLE 9i Release 2 | |
| 10.1 | JAN 2004 | ORACLE 10g Release 1 | g =Grid Computing (Multiple DB Servers can be connected Together) |
| 10.2 | JUL 2005 | ORACLE 10g Release 2 | |
| 11.1 | 2007 | ORACLE 11g Release 1 | |
| 11.2 | | ORACLE 11g Release 2 | |
| 11.3 | 2014 | ORACLE 12c Release 1 | c = Cloud , DB can be accessed from the cloud directly. |

## Different Databases

| Company | Database | Language |
|---|---|---|
| Oracle | Oracle | SQL , PL/SQL |
| Microsoft | SQL Server | SQL , T-SQL |
| IBM | DB2 | SQL, PL/SQL |
| SUN(Oracle) | MySQL | SQL, PL/SQL |
| Microsoft | Access | |
| | FoxPro | |

# Table (Intension & Extension)

**Intension**

- The intension of a given relation is independent of time.
- It is the permanent part of the relation.
- It corresponds to what is specified in the relational schema.
- The intension is a combination of two things : a structure and a set of integrity constraints.
- The naming structure consists of the relation name plus the names of the attributes (each with its associated domain name).
- The integrity constraints can be subdivided into key constraints, referential constraints, and other constraints.

Example: This is the intension of Employee relation.

```
Employee
(
EmpNo       Number(4)   Not NULL  ,
EName       varchar(20)  ,
Age         Number(2) ,
 Dept       varchar(4)
)
```

**Extension**

- The extension of a given relation is the set of tuples appearing in that relation at any given instance.
- The extension thus varies with time. It changes as tuples are created, destroyed, and updated.

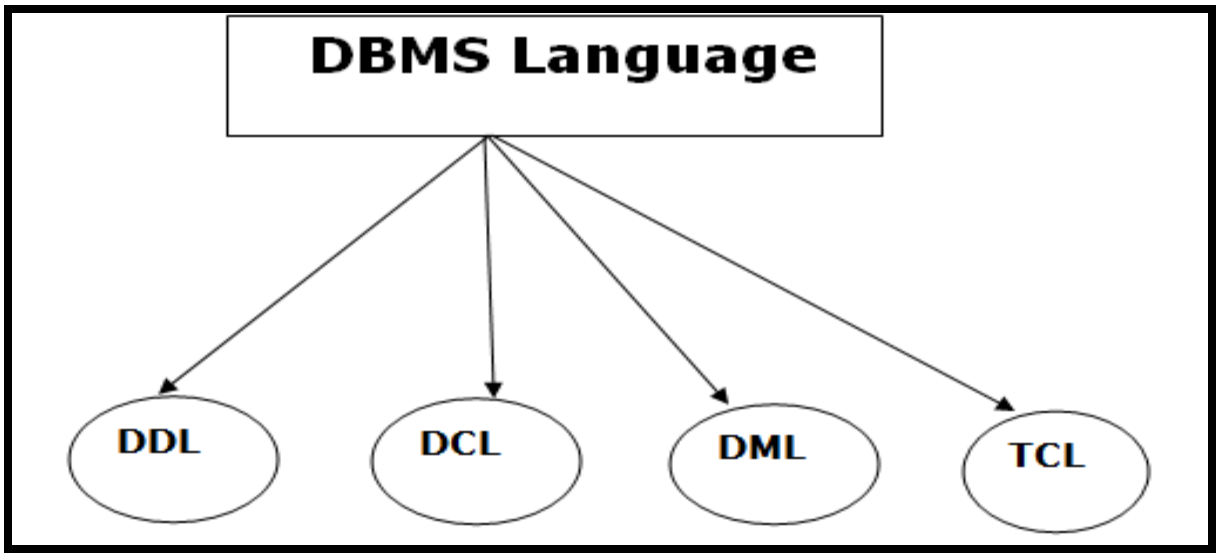
Relation: Employee at time= t1

| EmpNo | EmpName | Age | Dept |
|---|---|---|---|
| 1001 | Jason | 23 | SD |
| 1002 | William | 24 | HR |
| 1003 | Jonathan | 28 | Fin |
| 1004 | Harry | 20 | Fin |

Relation: Employee at time= t2 after adding more records

| EmpNo | EmpName | Age | Dept |
|---|---|---|---|
| 1001 | Jason | 23 | SD |
| 1002 | William | 24 | HR |
| 1003 | Jonathan | 28 | Fin |
| 1004 | Harry | 20 | Fin |
| 1005 | Smith | 22 | HR |
| 1006 | Mary | 19 | HR |
| 1007 | Sarah | 23 | SD |

# Database Language




## 1. Data Definition Language

**DDL** Commands are used to Create, Modify and Delete Database Structure but not the Data. DDL Commands are:

- **Create:** It is used to create objects in the database.
- **Alter:** It is used to alter the structure of the database.
- **Drop:** It is used to delete objects from the database.
- **Truncate:** It is used to remove all records from a table.
- **Rename:** It is used to rename an object.
- **Comment:** It is used to comment on the data dictionary.

## 2. Data Manipulation Language

**DML** stands for **D**ata **M**anipulation **L**anguage. It is used for accessing and manipulating data in a database. It handles user requests. DML Commands are:

- **Select:** It is used to retrieve data from a database.
- **Insert:** It is used to insert data into a table.
- **Update:** It is used to update existing data within a table.
- **Delete:** It is used to delete all records from a table.
- **Merge:** It performs UPSERT operation, i.e., insert or update operations.

## 3. Data Control Language

**DCL** stands for **D**ata **C**ontrol **L**anguage. It Controls Access to Data and to the Database.

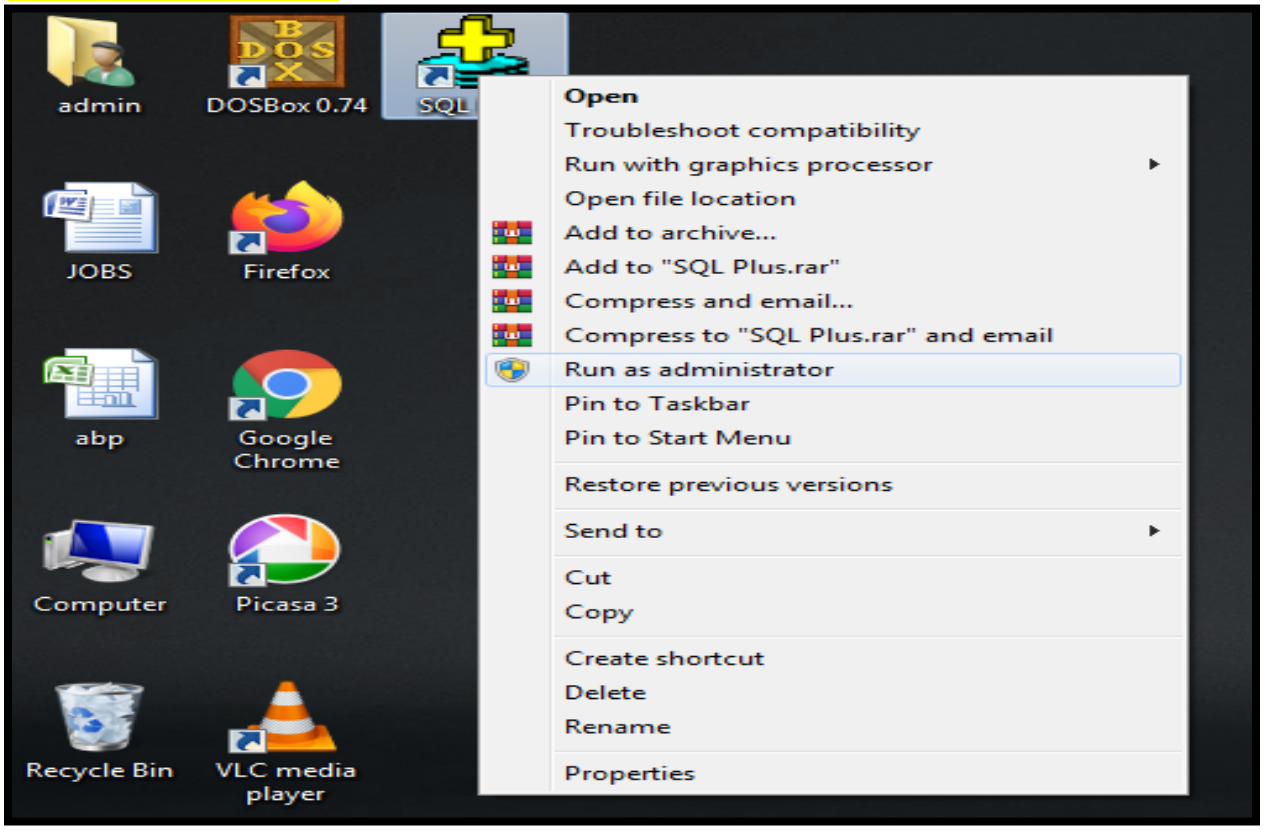
- **Grant:** It is used to give user access privileges to a database.
- **Revoke:** It is used to take back permissions from the user.

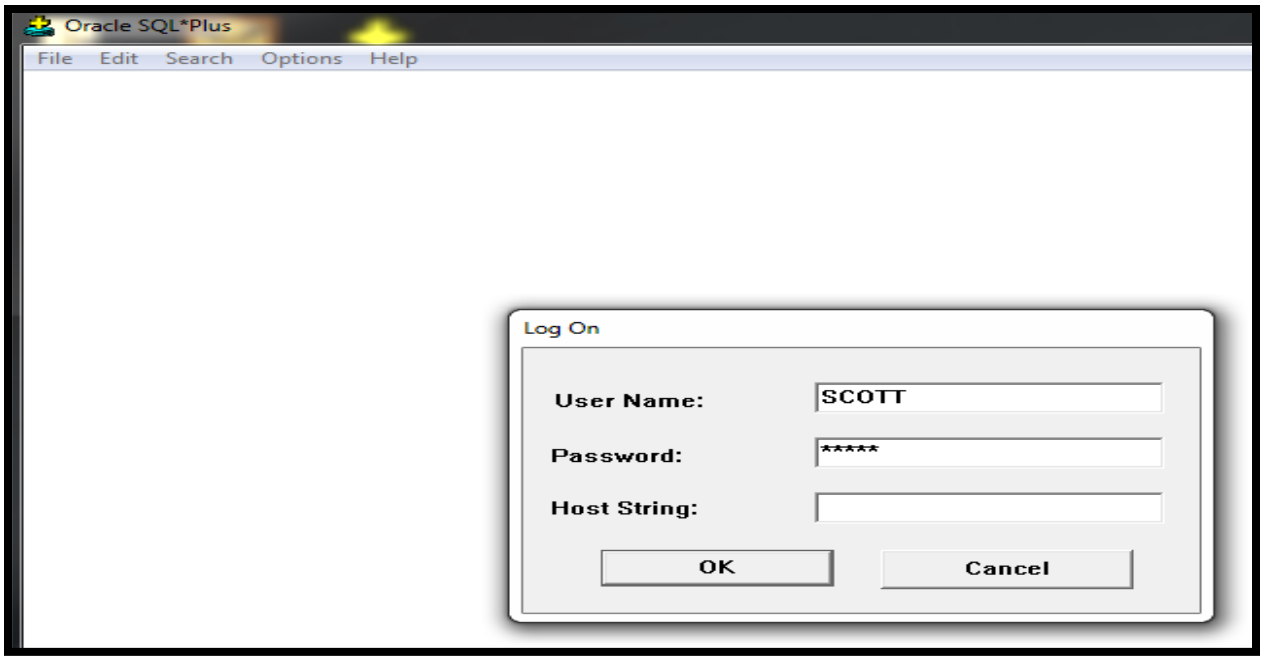
### 4. Transaction Control Language

**TCL** is used to run the changes made by the DML statement. TCL can be grouped into a logical transaction.

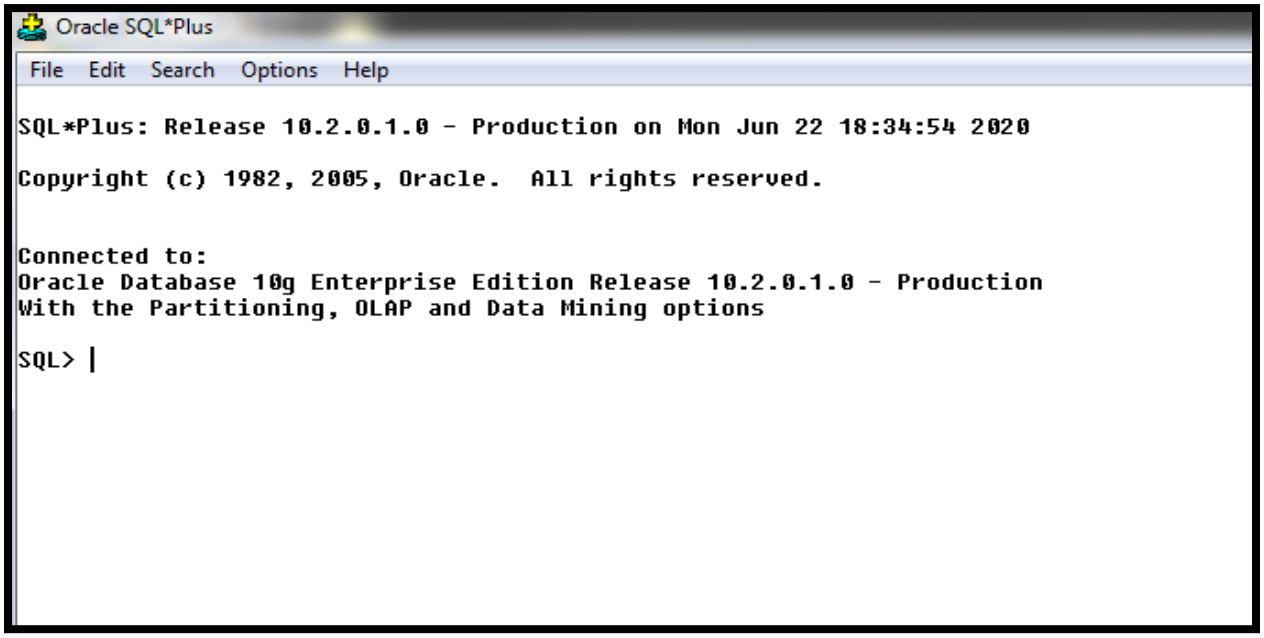
- **Commit:** It is used to save the transaction on the database.
- **Rollback:** It is used to restore the database to original since the last Commit.

## STARTING ORACLE






**USER NAME - SCOTT PASSWORD - tiger**




==========

## TO VIEW LIST OF TABLES AVAILABLE IN DATABASE

```
SQL>  SELECT * FROM TAB;

TNAME                          TABTYPE  CLUSTERID
------------------------------ ------- ----------
DEPT                           TABLE
EMP                            TABLE
BONUS                          TABLE
SALGRADE                        TABLE
EMP_MSTR                       TABLE
```

## CREATE TABLE

**Syntax-**

```
create table <table name>
(
column1    datatype    (width),
column2    datatype    (width),
 --------------
);
```

```
SQL> create table EMP_MSTR
(       EMPNO       VARCHAR    (4),
        ENAME       VARCHAR    (20),
        DEPT        VARCHAR    (8),
        DOJ         DATE,
        SAL         NUMBER     (6)
);
```

**Table created.**

# TO SEE THE STRUCTURE OF THE CREATED TABLE

**DESC – DESCRIBE**

```
SQL> DESC EMP_MSTR;
```

| Name | Null? | Type |
|---|---|---|
| EMPNO | | VARCHAR2(4) |
| ENAME | | VARCHAR2(20) |
| DEPT | | VARCHAR2(8) |
| DOJ | | DATE |
| SAL | | NUMBER(6) |

# TO INSERT VALUES INTO CREATED TABLE

**Syntax 1-**

insert into <table name> (column1, column2, --------column n)
values(value1, value2,-------- value n);

**Query 1-**

```
SQL> insert into EMP_MSTR (EMPNO, ENAME,DEPT,DOJ, SAL)
VALUES ('E1','AMAN JAIN','ACCOUNT','10-JAN-2018',20000);
```

**Output-** 1 row(s) inserted.

**Syntax 2-**

insert into <table name> (column1, column2, --------column n)
values('&COLUMN NAME', ……..);

```
SQL> INSERT INTO EMP_MSTR VALUES ('&EMPNO' , '&ENAME' , '&DEPT' ,
'&DOJ' , &SAL);
```

**Enter value for empno:** E2
**Enter value for ename:** PRIYA SINGH
**Enter value for dept:** TPO
**Enter value for doj:** 15-FEB-2018
**Enter value for sal:** 25000

**Output- 1 row created.**

## To view the Created Record

**SQL> select * from EMP_MSTR;**

| EMPN | ENAME | DEPT | DOJ | SAL |
|---|---|---|---|---|
| E1 | PRIYA SINGH | TPO | 15-FEB-18 | 25000 |

==========

## To Create Table

```
sql> create table student
       (name varchar2(15), course varchar2(8),
       branch varchar2(5), fees number(6)
       );
Table created
```

## To See the Structure of the Table (DESC - Describe)

```
SQL>DESC student;
table structute
```

## To Insert a Row in Table

```
sql> insert into student values
       ('&name' , '&course' , '&branch' , &fees);
1 row created

sql> /
```

## TO Insert Multiple Rows in a Table / in 2 Table @ a Time

```
sql>create table TB1 (id number, name varchar(20));

Table Created

sql>create table TB2 (id number, name varchar(20));

Table Created

Sql> Insert All
      Into TB1 Values(    1 , ‘ RAM’)
      Into TB1 Values(    2 , ‘ SHYAM’)
      Into TB1 Values(    3, ‘  DEEPAK’)
      SELECT * FROM Dual;

3 Rows created

Sql>select * from TB1;
      ID          Name
--------------------------------
      1           RAM
      2           SHYAM
      3           DEEPAK
Sql> Insert All
      Into TB1 Values(    4, ‘ ABC’)
      Into TB1 Values(    5, ‘ DEF’)
      Into TB2 Values(    1 , ‘ RAM’)
      Into TB3 Values(    2 , ‘ SHYAM’)
      Into TB4 Values(    3, ‘  DEEPAK’)
      SELECT * FROM Dual;

3 Rows created
```

## SELECT

### To Select All Tables of the Data Base

```
sql>  select * from tab;
```

### To Select All Rows / Columns of the Table

```
Sql> select * from student;

Sql> select * from EMP;
```

### To View Selected Columns of the Table

```
sql> select ENAME, SAL FROM EMP;

sql> select ENAME, HIREDATE, SAL FROM EMP;
```

### To View Selected Columns of the Table based on Condition

```
sql> select ENAME, SAL FROM EMP
       WHERE SAL > = 20000;

sql> select ENAME, SAL , HIREDATE FROM EMP
WHERE HIREDATE >= '02-APR-20';

sql> select SUM(SAL) FROM EMP;
```

## UPDATE CLAUSE

```
sql> UPDATE     EMP
     SET        SAL      = 1000
     WHERE      ENAME    = 'SMITH';

sql> UPDATE     EMP
     SET        ENAME    = ‘SMITH’ ,
                SAL      = 20000
     WHERE      ENAME    = 'ABCD';

sql> UPDATE     EMP
     SET        ENAME    = ‘SMITH’ ,
                SAL      = 20000
     WHERE      ENAME     = 'ABCD' ,
                SAL      = 10000;

sql> UPDATE     student
     SET        mobile   = 9878654523
     WHERE      name      = 'arjun';
```

1 row updated

## ORDER BY CLAUSE / SORTING

ASC - Ascending Order
DESC - Descending Order

```
sql> select ENAME , SAL FROM EMP
ORDER BY ENAME ASC;

sql> select * FROM EMP
ORDER BY ENAME DESC;
```

```
SQL> SELECT * FROM DEPT
       ORDER BY LOC;

   DEPTNO    DNAME              LOC
--------- -------------- ------------ ------------------
       30         SALES              CHICAGO
       20         RESEARCH           DALLAS
       10         TP CELL            NEW YORK
```

**ALTER CLAUSE**

- For Adding Columns
- For Dropping Columns
- For Renaming Columns
- Hiding Columns
- Modifying Data Types (Size)
- Modifying Size of Columns (Increase / Decrease)
- Adding / Dropping Constraints
- Enabling / Disabling Constraints

Sql>Alter Table <Table Name>
Add
Modify
Drop
Rename

**Adding New Column in Table**

SQL>DESC EMP;

SQL> ALTER TABLE EMP
ADD DEPT VARCHAR2(10);

SQL>DESC EMP;

```
SQL> SELECT * FROM DEPT;

   DEPTNO       DNAME         LOC            MOBILE
--------- ------------ ------------ -----------------------------------
       10            TP CELL         NEW YORK
       20            RESEARCH      DALLAS
       30            SALES           CHICAGO

SQL> ALTER TABLE DEPT
        ADD (SALARY NUMBER(6));

Table altered.

SQL> SELECT * FROM DEPT;

   DEPTNO      DNAME         LOC         MOBILE     SALARY
--------- ------------ ------------ --------- ----------------------------
       10            TP CELL         NEW YORK
       20            RESEARCH     DALLAS
       30            SALES           CHICAGO

SQL> DESC DEPT;
 Name                      Null?                Type
 ----------------------------------------- -------- ---------------------
 DEPTNO                    NOT NULL         NUMBER(2)
 DNAME                                      VARCHAR2(14)
 LOC                                        VARCHAR2(13)
 MOBILE                                     NUMBER(12)
 SALARY                                     NUMBER(6)
```

**To Rename Table**

ALTER TABLE table_name
RENAME TO new_table_name;

```
SQL> select * from box1;
        ID      COLOR
---------- ----------
         1      Red
         2      Green
         3      Blue
         4      Purple

SQL> create table box3 as select * from box1;

Table created.

SQL> desc box3;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ID                                                 NUMBER(4)
 COLOR                                     NOT NULL VARCHAR2(10)

SQL> alter table box3
  2  rename to NEWBOX;

Table altered.

SQL> DESC BOX3;
ERROR:
ORA-04043: object BOX3 does not exist

SQL> DESC NEWBOX;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ID                                                 NUMBER(4)
 COLOR                                     NOT NULL VARCHAR2(10)
```

SQL> RENAME CUSTOMERS TO CUSTOMER;

Table renamed.

**To Drop Table**

**DROP -** Table will be permanently deleted from the Database.

```
SQL> DROP TABLE CUSTOMER;

Table dropped.

SQL> SELECT * FROM CUSTOMER;
SELECT * FROM CUSTOMER
              *
ERROR at line 1:
ORA-00942: table or view does not exist
```

**To Rename Column Name inTable**

```
ALTER TABLE table_name
RENAME COLUMN old_name TO new_name;
```

```
SQL> DESC NEWBOX;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ID                                                 NUMBER(4)
 COLOR                                     NOT NULL VARCHAR2(10)

SQL> ALTER   TABLE    NEWBOX
         RENAME       COLUMN ID TO CODE ;

Table altered.
```

```
SQL> DESC NEWBOX;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CODE                                               NUMBER(4)
 COLOR                                     NOT NULL VARCHAR2(10)
```

**PRIMARY KEY**

```
SQL> alter table dept
 drop (deptno);

drop (deptno)
      *
ERROR at line 2:
cannot drop parent key column
```

**DELETE**

**Deleting All Records**

```
sql> DELETE FROM TABLE_NAME;
sql> DELETE [FROM] TABLE_NAME;

sql> DELETE FROM EMP;
```

**Deleting Record Based on Condition**

```
sql> DELETE FROM EMP
WHERE ENAME = 'SMITH';

sql> DELETE FROM EMP
```

WHERE DOB = '12-MAR-05';

```
SQL> SELECT * FROM EMPLOYEE;
   EMPID       EMPNAME                          DEPTID
--------- ---------------------------------- ----------
       1            HERRY                        2
       2            TOM                             3
       3            JOY                             5
       4            ROY                             8

SQL> DELETE FROM EMPLOYEE
       WHERE EMPID = 1 AND DEPTID = 2;
1 row deleted.

SQL> SELECT * FROM EMPLOYEE;
   EMPID       EMPNAME                          DEPTID
--------- ---------------------------------- ----------
       2            TOM                             3
       3            JOY                             5
       4            ROY                             8
```

## Group by Clause

In Oracle GROUP BY clause is used with SELECT statement to collect data from multiple records and group the results by one or more columns.

**Example 1** - **Entity** - SALESDEPARTMENT

| ITEM | SALE | BILLING_ADDRESS |
|---|---|---|
| Shoes | 120 | Agra |
| Belts | 105 | Kolkata |
| Shoes | 45 | Allahabad |
| Sari | 210 | Varanasi |
| Sari | 5000 | Chennai |
| Medicines | 250 | Salem |
| Computer | 210 | Delhi |
| Shoes | 1000 | Kanpur |

**Query -** Show the total sales of every individual Item .

```
SQL>SELECT item , SUM(sale) AS  "Total sales"
        FROM SALESDEPARTMENT
        GROUP BY item;
```

**Output**

| ITEM | Total Sales |
|---|---|
| Belts | 105 |
| Sari | 5210 |
| Shoes | 1165 |
| Medicines | 250 |
| Computer | 210 |

**Example 2 (Group by with COUNT function)** Entity - **CUSTOMERS**

| NAME | AGE | SALARY | STATE |
|---|---|---|---|
| mohan | 16 | 12000 | bihar |
| rohan | 21 | 25000 | bihar |
| tejpratap | 26 | 56000 | bihar |
| chanchal | 21 | 25000 | delhi |
| charu | 21 | 23000 | delhi |
| chanda | 27 | 27000 | delhi |
| chandni | 31 | 31000 | delhi |
| pralaynath | 56 | 76000 | tamilnadu |
| gundaswami | 65 | 87000 | tamilnadu |
| bina | 18 | 18000 | maharashtra |

**Query -** Show the total N o. of Customers in a City.

```
SQL>  SELECT state, COUNT(*) AS "Number of customers"
      FROM customers
      WHERE salary > 10000
      GROUP BY state;
```

Output:

| STATE | Number Of Customers |
|---|---|
| maharashtra | 1 |
| delhi | 4 |
| tamilnadu | 2 |
| bihar | 3 |

## LIKE OPERATOR / LIKE PATTERN

Using Like Pattern we can select matching records.
The pattern can contain 2 wild cards characters –

1. **%** (Percentage )
2. **_** (Underscore )

The % (Percentage) is used to denote any number of unknown characters and _ (Underscore) denote one unknown character.

**Example 1:** Find out the Salary details of the Employees, whose name contains L as 2nd Letter **.**

```
Sql> select ENAME, sal from EMP
where ename like '_L%';
```

**Example 2:** Find out the Salary details of the Employees, whose name starts with Letter 'A'.

```
Sql> select ENAME, sal from EMP
where ename like 'A%';
```

## IN OPERATOR

Using IN we can specify a list of possible values for any column.

**Example 1:** Select employee list whose designation is 'Admin' or 'Hod' or 'Faculty'

```
Sql> select * from EMP where Desig IN ('Admin' , 'Hod' ,
'Faculty');
```

OR

```
Sql> select     *      from      EMP
     where      Desig = 'Admin' OR
                Desig = 'Hod' OR
                Desig = 'Faculty';
```

## BETWEEN OPERATOR

Using BETWEEN we can select all the records which fall in selected range.

**Example 1:** Select all records between id 1002 and 1007.

```
Sql> select     *      from      STUDENT
     where      ID   BETWEEN  1002 AND 1005;
```

OR

```
Sql> select     *      from      STUDENT
     where      ID > = 1002 AND  ID < = 1005;
```

**Example 2:** Select all records between DOB Date TO Date .

```
Sql> select     *      from      STUDENT
     where      DOB       BETWEEN
     '05-JUN-2002'       AND       '10-JUL-2003';
```

# SAVE POINT, ROLLBACK, COMMIT

**Savepoint - Set a location or point**
**Roll back - Undo (Create / Insert / Delete / Update)**
**Commit - Permanently save data**

```
SQL> savepoint s1;
Savepoint created.

SQL> create table contact(id number, name varchar(20));
Table created.

SQL> insert into contact values(&id,'&name');
Enter value for id:             1
Enter value for name:           RAM
1 row created.

SQL> /
Enter value for id:             2
Enter value for name:           RAMESH
1 row created.

SQL> select * from contact;
        ID NAME
---------- --------------------
         1 RAM
         2 RAMESH

SQL> update contact set name = 'deepak'  where id = 1;
1 row updated.
```

```
SQL> select * from contact;
        ID          NAME
---------- --------------------
         1          DEEPAK
         2          RAMESH

SQL> delete from contact where id = 1;
1 row deleted.

SQL> select * from contact;
        ID          NAME
---------- --------------------
         2          RAMESH

SQL> roll back to s1;
Rollback complete.

SQL> select * from contact;
No Rows Selected

--------------------------------------------------------------------------------
SQL> savepoint s2;
Savepoint created.

SQL> insert into contact values(&id , '&name');
Enter value for id:             1
Enter value for name:           RAM
1 row created.

sql> select * from contact;
        ID          NAME
---------- --------------------
         1          RAM
```

```
SQL> commit;
Commit complete.

SQL> roll back s2;
Rollback complete.

SQL> select * from contact;
        ID NAME
---------- --------------------
         1 RAM
```

# REFERENTIAL INTEGRITY

## Master Table

STUDENT_ADD

| ROLL_NO | NAME | CITY | MOBILE | PIN |
|---|---|---|---|---|
| 1 | ABHIMANYU KUMAR | BOKARO | 5674568976 | 827001 |
| 2 | DEEPAK GUPTA | DHNBAD | 5674568976 | 827002 |
| 3 | RAMESH KUMAR | DEOGHAR | 5674568976 | 827004 |
| 4 | NAVEEN KUMAR | RANCHI | 5674568976 | 827006 |
| 5 | RAVI SHANKAR | RANCHI | 5674568976 | 827007 |

## Child Table

STUDENT_MARKS

| ROLL_NO | SUBJECT | MARKS |
|---|---|---|
| 1 | VISUAL BASIC | 78 |
| 2 | VISUAL BASIC | 69 |
| 2 | ORACLE | 81 |
| 3 | C++ | 92 |

```
SQL> create table student_add(
    roll_no number primary key,
    name varchar(20),
    city varchar(10),
    mobile varchar(10),
    pin varchar(6)
    );
```

**Table created.**

```
SQL> create table student_marks(
   roll_no number references student_add,
   subject varchar(15),
   marks number(2)
   );

Table created.

SQL> desc student_add;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLL_NO                                   NOT NULL NUMBER
 NAME                                               VARCHAR2(20)
 CITY                                               VARCHAR2(10)
 MOBILE                                             VARCHAR2(10)
 PIN                                                VARCHAR2(6)

SQL> desc student_marks;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLL_NO                                            NUMBER
 SUBJECT                                            VARCHAR2(15)
 MARKS                                              NUMBER(2)

SQL> insert into student_add values
(&ROLL_NO,'&NAME','&CITY','&MOBILE','&PIN');

SQL> select * from student_add;

  ROLL_NO NAME                 CITY       MOBILE     PIN
--------- -------------------- ---------- ---------- ------
        1 abhimanyu kumar      bhopal     8967452319 876543
        2 deepak gupta         indore     9876543219 987654
```

**3 ramesh kumar delhi 9876543210 987654**
**4 navin kumar jabalpur 9876543210 987654**
**5 ravi shankar bhopal 9876543210 987654**

**SQL> insert into student_marks values(&rollo_no,'&subject',&marks);**

**SQL> select * from student_marks;**

| ROLL_NO | SUBJECT | MARKS |
|---|---|---|
| 1 | c++ | 89 |
| 1 | java | 78 |
| 2 | toc | 79 |

**To Solve Insert Anomally**

SQL> insert into student_marks
values(&rollo_no,'&subject',&marks);
Enter value for rollo_no: 6
Enter value for subject: BCE
Enter value for marks: 88
old 1: insert into student_marks
values(&rollo_no,'&subject',&marks)
new 1: insert into student_marks values(6,'BCE',88)
insert into student_marks values(6,'BCE',88)
*
ERROR at line 1:
integrity constraint -violated - parent key not
found

## To Solve Delete Anomally

```
SQL> create table student_marks(
   roll_no number references student_add on delete cascade,
   subject varchar(15),
   marks number(2)
   );

Table crated

SQL> insert into student_marks
values(&rollo_no,'&subject',&marks);

SQL> select * from student_marks;

  ROLL_NO        SUBJECT          MARKS
---------- --------------- ---------------------------
         1                 c++                      89
         1                 java                     78
         2                 toc                      79

Sql> delete from student_add
Where roll_no = 1;

1 Row Deleted

SQL> select * from student_marks;

  ROLL_NO        SUBJECT          MARKS
---------- --------------- -----------------------------------
         2                 toc                      79
```

## Column_Alias

It gives desired Column Name in the Resultant Table.

```
SQL> select * from dept;

   DEPTNO    DNAME              LOC            SALARY
--------- -------------- ------------ ---------- ----------------------
       10          TP CELL            NEW YORK         45000
       20          RESEARCH           DALLAS           50000
       30          SALES              CHICAGO          55000
```

```
SQL> select deptno "Dept Number" ,
       Dname "Dept Name"
       from dept;
```

```
Dept Number            Dept Name
----------- ----------------------------------
         10                TP CELL
         20                RESEARCH
         30                SALES
```

## Table_Alias

It is used for Table Joining.

```
SQL> SELECT * FROM newdept;

      DNO            DNAME
--------- ------------------------------
         10              MCA
         20              MBA
         30              BCA
         40              MTECH
```

SQL> SELECT * FROM newemp;

| ENO | NAME | DNO |
|---|---|---|
| 1 | SWARA | 10 |
| 2 | SANVI | 20 |
| 3 | SAHAJ | 10 |
| 4 | PARAM | 50 |

Ex: From the Tables dept and emp , Name of the employees in various Dept.

```
SQL> SELECT NAME,DNAME FROM
NEWEMP,NEWDEPT
WHERE NEWEMP.DNO = NEWDEPT.DNO;
```

| NAME | DNAME |
|---|---|
| SAHAJ | MCA |
| SWARA | MCA |
| SANVI | MBA |

**Table Alias -**

```
SQL> SELECT NAME,DNAME FROM NEWEMP E ,
NEWDEPT D
 WHERE E.DNO = D.DNO;
```

| NAME | DNAME |
|---|---|
| SAHAJ | MCA |
| SWARA | MCA |
| SANVI | MBA |

**Equi join**

- Uses Equal Operator (=)

```
SQL> SELECT ENO,NAME,DNAME FROM
NEWEMP,NEWDEPT
 WHERE NEWEMP.DNO = NEWDEPT.DNO;

       ENO NAME       DNAME
---------- ---------- --------------------------------
         3 SAHAJ      MCA
         1 SWARA      MCA
         2 SANVI      MBA
```

**Non Equi join**

- Uses any other operator apart from Equal (=)
- > < >= <= !=

```
SQL> SELECT * FROM newdept;

       DNO            DNAME
---------- --------------------------------
        10            MCA
        20            MBA
        30            BCA
        40            MTECH

SQL> SELECT * FROM newemp;

       ENO            NAME          DNO
---------- ---------- ----------------------------------
         1            SWARA              10
         2            SANVI              20
         3            SAHAJ              10
         4            PARAM              50
```

```
SQL> SELECT ENO,NAME,DNAME FROM
NEWDEPT,NEWEMP
  WHERE NEWDEPT.DNO < NEWEMP.DNO;

       ENO NAME       DNAME
---------- ---------- ----------
         2 SANVI      MCA
         4 PARAM      MCA
         4 PARAM      MBA
         4 PARAM      BCA
         4 PARAM      MTECH


SQL> SELECT ENO,NAME,DNAME FROM
NEWDEPT,NEWEMP
  2  WHERE NEWDEPT.DNO > NEWEMP.DNO;

       ENO NAME       DNAME
---------- ---------- ----------
         2 SANVI      MTECH
         3 SAHAJ      MTECH
         1 SWARA      MTECH
         2 SANVI      BCA
         3 SAHAJ      BCA
         1 SWARA      BCA
         3 SAHAJ      MBA
         1 SWARA      MBA

8 rows selected.
```

## SELF JOIN

The SQL **SELF JOIN** is used to **join** a table to itself as if the table were two tables;

SQL> SELECT * FROM newemp;

| ENO | NAME | DNO |
|---|---|---|
| 1 | SWARA | 10 |
| 2 | SANVI | 20 |
| 3 | SAHAJ | 10 |
| 4 | PARAM | 50 |

**SQL> SELECT E1.ENO , E2.NAME , E1.DNO**
**2 FROM NEWEMP E1 , NEWEMP E2**
**3 WHERE E1.DNO = E2.DNO;**

| ENO | NAME | DNO |
|---|---|---|
| 3 | SWARA | 10 |
| 1 | SWARA | 10 |
| 2 | SANVI | 20 |
| 3 | SAHAJ | 10 |
| 1 | SAHAJ | 10 |
| 4 | PARAM | 50 |

**6 rows selected.**

## NATURAL JOIN

A **NATURAL JOIN** is a **JOIN** operation that creates an implicit **join** clause for you based on the common columns in the two tables being joined. Common columns are columns that have the same name in both tables. A **NATURAL JOIN** can be an INNER **join**, a LEFT OUTER **join**, or a RIGHT OUTER **join**. The default is INNER **join**.

SQL> SELECT * FROM newdept;

| DNO | DNAME |
|---|---|
| 10 | MCA |
| 20 | MBA |
| 30 | BCA |
| 40 | MTECH |

SQL> SELECT * FROM newemp;

| ENO | NAME | DNO |
|---|---|---|
| 1 | SWARA | 10 |
| 2 | SANVI | 20 |
| 3 | SAHAJ | 10 |
| 4 | PARAM | 50 |

**SQL> SELECT ENO, NAME, DNO , DNAME FROM NEWEMP NATURAL JOIN NEWDEPT;**

| ENO | NAME | DNO | DNAME |
|---|---|---|---|
| **3** | **SAHAJ** | **10** | **MCA** |
| **1** | **SWARA** | **10** | **MCA** |
| **2** | **SANVI** | **20** | **MBA** |

SQL> SELECT DNO , DNAME , ENO , NAME  FROM NEWDEPT NATURAL JOIN NEWEMP;

| DNO | DNAME | ENO | NAME |
|---|---|---|---|
| 10 | MCA | 1 | SWARA |
| 20 | MBA | 2 | SANVI |
| 10 | MCA | 3 | SAHAJ |

**CROSS JOIN**

Cross Multiplication of Rows.

SQL> select * from newdept , newemp;

**Or**

SQL> select * from newdept cross join newemp;

| DNO | DNAME | ENO | NAME | DNO |
|---|---|---|---|---|
| 10 | MCA | 1 | SWARA | 10 |
| 20 | MBA | 1 | SWARA | 10 |
| 30 | BCA | 1 | SWARA | 10 |
| 40 | MTECH | 1 | SWARA | 10 |
| 10 | MCA | 2 | SANVI | 20 |
| 20 | MBA | 2 | SANVI | 20 |
| 30 | BCA | 2 | SANVI | 20 |
| 40 | MTECH | 2 | SANVI | 20 |
| 10 | MCA | 3 | SAHAJ | 10 |
| 20 | MBA | 3 | SAHAJ | 10 |
| 30 | BCA | 3 | SAHAJ | 10 |
| 40 | MTECH | 3 | SAHAJ | 10 |
| 10 | MCA | 4 | PARAM | 50 |

| | | | | |
|---|---|---|---|---|
| 20 | MBA | 4 | PARAM | 50 |
| 30 | BCA | 4 | PARAM | 50 |
| 40 | MTECH | 4 | PARAM | 50 |

**16 rows selected.**

## OUTER JOIN

**Outer Join is a join that can be used to return the records in both the tables whether it has matching records in both the tables or not.**

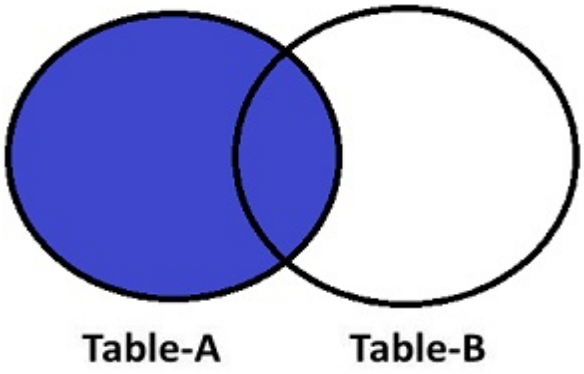
***The outer join can be further divided into three types:***

1. **LEFT**
2. **RIGHT**
3. **OUTER**

**1. Left-Outer Join:**

**The Left-Outer Join is an outer join that returns all the values of the left table, and the values of the right table that has matching values in the left table.** If there is no matching result in the right table, it will return null values in that field. The Left-Outer Join can be depicted using the below diagram.

- **All Rows From Left Table & Only Matching Rows From Right Table.**



**SQL> SELECT * FROM NEWEMP , NEWDEPT**
**WHERE NEWEMP.DNO = NEWDEPT.DNO (+);**

**OR**

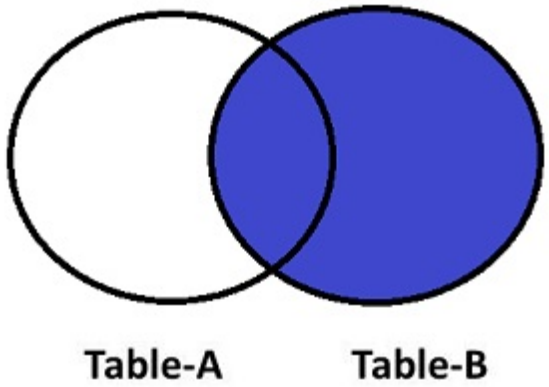
**SQL> SELECT * FROM NEWEMP LEFT OUTER JOIN NEWDEPT**
**ON NEWEMP.DNO = NEWDEPT.DNO;**

| ENO | NAME | DNO | DNO | DNAME |
|---|---|---|---|---|
| 3 | SAHAJ | 10 | 10 | MCA |
| 1 | SWARA | 10 | 10 | MCA |
| 2 | SANVI | 20 | 20 | MBA |
| 4 | PARAM | 50 | | |

**2. Right-Outer Join:**

**The Right-Outer Join is an outer join that returns all the values of the right table, and the values of the left table that has matching values in the right table.** The Right-Outer Join can be depicted using the below diagram.

- **All Rows From Right Table & Only Matching Rows From Left Table.**



**SQL> SELECT * FROM NEWEMP , NEWDEPT**

**WHERE NEWEMP.DNO (+) = NEWDEPT.DNO;**

**OR**

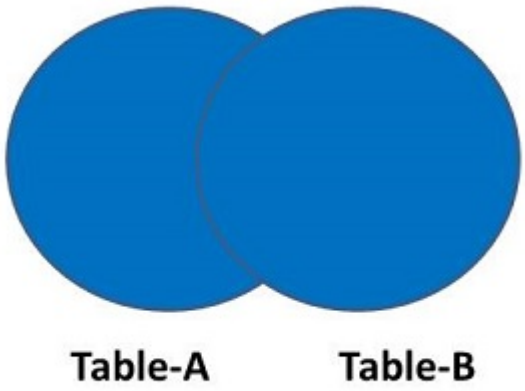
**SQL> SELECT * FROM NEWEMP RIGHT OUTER JOIN NEWDEPT**

**ON NEWEMP.DNO = NEWDEPT.DNO;**

| ENO | NAME | DNO | DNO | DNAME |
|---|---|---|---|---|
| 1 | SWARA | 10 | 10 | MCA |
| 2 | SANVI | 20 | 20 | MBA |
| 3 | SAHAJ | 10 | 10 | MCA |
| | | | 30 | BCA |
| | | | 40 | MTECH |

**3. Full-Outer Join:**

**The Full-Outer join contains all the values of both the tables whether they have matching values in them or not.** The Full-Outer Join can be depicted using the below diagram.

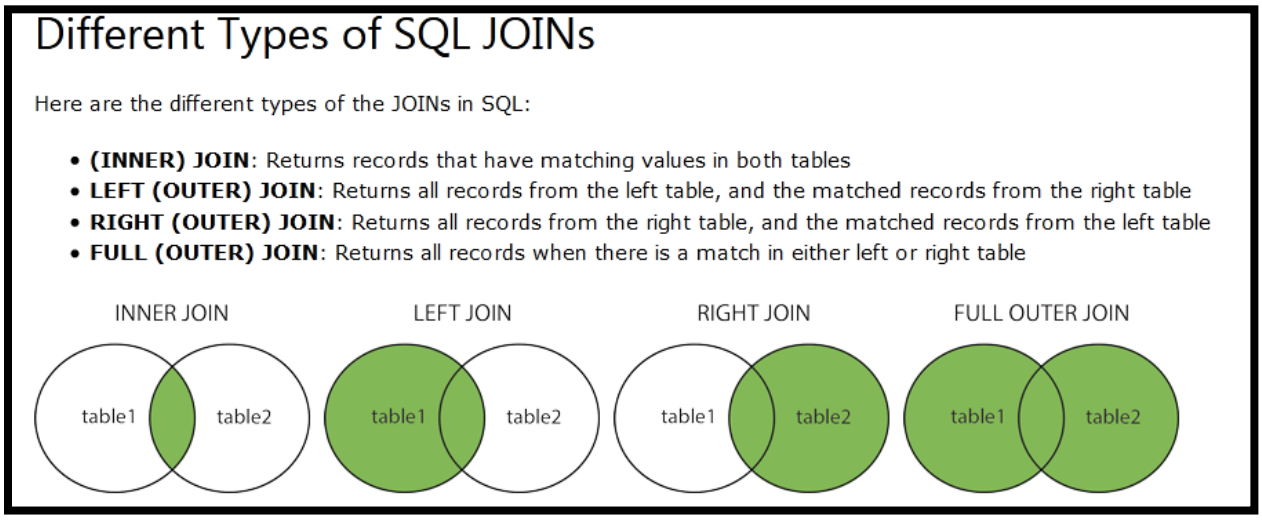



**SQL> SELECT * FROM NEWEMP FULL OUTER JOIN NEWDEPT**

**ON NEWEMP.DNO = NEWDEPT.DNO;**

| ENO | NAME | DNO | DNO | DNAME |
|---|---|---|---|---|
| 3 | SAHAJ | 10 | 10 | MCA |
| 1 | SWARA | 10 | 10 | MCA |
| 2 | SANVI | 20 | 20 | MBA |
| 4 | PARAM | 50 | | |
| | | | 30 | BCA |
| | | | 40 | MTECH |

**6 rows selected.**



# FUNCTIONS

## Link

https://www.techonthenet.com/oracle/functions/index.php

## Function Types

**Numeric/Math Functions**

| | | |
|---|---|---|
| ABS | FLOOR | ROUND (numbers) |
| ACOS | GREATEST | ROWNUM |
| ASIN | LEAST | SIGN |
| ATAN | LN | SIN |
| ATAN2 | LOG | SINH |
| AVG | MAX | SQRT |
| BITAND | MEDIAN | SUM |
| CEIL | MIN | TAN |
| COS | MOD | TANH |
| COSH | POWER | TRUNC (numbers) |
| COUNT | REGEXP_COUNT | |
| EXP | REMAINDER | |

### Aggregate Functions in Oracle

| | |
|---|---|
| AVG(x) | It returns average of any numeric column.<br>SELECT AVG(SALARY) FROM EMPLOYEE; |
| COUNT(x) | It returns total number records in any column.<br>SELECT COUNT(SALARY) FROM EMPLOYEE;<br>SELECT COUNT(NAME) FROM EMPLOYEE; |
| COUNT(*) | It returns total number of records in the table.<br>SELECT COUNT(*) FROM EMPLOYEE; |
| COUNT(DISTINCT x) | It returns total number of distinct records in any column.<br>SELECT COUNT(DISTINCT name) FROM EMPLOYEE; |
| MAX(x) | It returns maximum value in any column.<br>SELECT MAX(SALARY) FROM EMPLOYEE; |
| MAX(DISTINCT x) | It returns maximum distinct value in any column. |
| MIN(x) | It returns minimum value in any column.<br>SELECT MIN(SALARY) FROM EMPLOYEE; |
| MIN(DISTINCT x) | It returns minimum distinct value in any column. |
| SUM(x) | It returns total sum of any column.<br>SELECT SUM(SALARY) FROM EMPLOYEE; |
| SUM(DISTINCT x) | It returns total sum of distinct values of any column. |

```
SQL> SELECT ENAME , SAL FROM EMP;

ENAME                          SAL
---------- ----------------------------------
SMITH                  800
ALLEN                  1600
WARD                   1250
JONES                  2975
MARTIN                 1250
BLAKE                  2850
CLARK                  2450
SCOTT                  3000
KING                        5000
TURNER                      1500
ADAMS                  1100
JAMES                  950
FORD                   3000
MILLER                 1300

14 rows selected.

SQL> SELECT AVG(SAL) FROM EMP;
  AVG(SAL)
----------
2073.21429
```

## COLUMN ALIAS

```
SQL> SELECT AVG(SAL) "AVERAGE SALARY OF EMPLOYEE"
FROM EMP;

AVERAGE SALARY OF EMPLOYEE
--------------------------------------------------
              2073.21429
```

```
SQL> SELECT  COUNT(SAL) FROM EMP;

COUNT(SAL)
----------
        14

SQL> SELECT  COUNT(*) FROM EMP;

  COUNT(*)
----------
        14

SQL> SELECT COUNT(DISTINCT SAL) FROM EMP;

COUNT(DISTINCTSAL)
------------------
                12

SQL> SELECT MAX(SAL) FROM EMP;

  MAX(SAL)
----------
      5000

SQL> SELECT SUM(SAL) FROM EMP;

  SUM(SAL)
----------
     29025
```

# String / Character Functions

| String/Char Functions | | |
|---|---|---|
| ASCII | INSTR4 | REGEXP_INSTR |
| ASCIISTR | INSTRB | REGEXP_REPLACE |
| CHR | INSTRC | REGEXP_SUBSTR |
| COMPOSE | LENGTH | REPLACE |
| CONCAT | LENGTH2 | RPAD |
| Concat with \|\| | LENGTH4 | RTRIM |
| CONVERT | LENGTHB | SOUNDEX |
| DECOMPOSE | LENGTHC | SUBSTR |
| DUMP | LOWER | TRANSLATE |
| INITCAP | LPAD | TRIM |
| INSTR | LTRIM | UPPER |
| INSTR2 | NCHR | VSIZE |

## Character Functions in Oracle

| | |
|---|---|
| INITCAP(data) | It converts first letter of string to uppercase.<br>SELECT INITCAP('ABHIMANYU') FROM DUAL;<br>Output: Abhimanyu |
| LENGTH(data) | It returns the length of the string.<br>SELECT LENGTH('i am abhimanyu') FROM DUAL;<br>Output: 14 |
| SUBSTR(data,x,y) | It returns substring from string.<br>SELECT SUBSTR('i am abhimanyu',3,2) FROM DUAL;<br>Output: am |
| INSTR(data,x) | It will find location of string 'x' in 'data' string.<br>SELECT INSTR('i am abhimanyu','am') FROM DUAL;<br>Output: 3 |
| GREATEST(n1,n2,n3,..) | It returns greatest number from group of numbers.<br>SELECT GREATEST(56,34,2,45,57) FROM DUAL;<br>Output: 57 |
| LEAST(n1,n2,n3,..) | It returns smallest number from group of numbers.<br>SELECT GREATEST(56,34,2,45,57) FROM DUAL;<br>Output: 2 |

- INITCAP

SQL> SELECT INITCAP('nikki') from dual;

INITCAP
---
Nikki
SQL> SELECT INITCAP(ename)from emp;

INITCAP(EN
---
Smith
Allen
Ward
Jones
Martin
Blake
Clark
Scott
King
Turner
Adams
James
Ford
Miller

14 rows selected.

## - length

SQL> select length(' hello ') from dual;

LENGTH ('HELLO')
---
7

- Substr(Data , x , y)
Y Character Starting from X Length

```
SQL> select substr('manipal university' , 5,6) from dual;

SUBSTR
------
pal un

SQL> select instr('i am a student','m') from dual;

- INSTR
INSTR('IAMASTUDENT','M')
------------------------
                       4

- greatest
SQL> select greatest(56,67,45,78) from dual;

GREATEST(56,67,45,78)
---------------------
                  78
- least
SQL> select least(56,67,45,78) from dual;

LEAST(56,67,45,78)
------------------
                45
```

## Number Function

### Number Functions in Oracle

| | |
|---|---|
| ABS(data) | It returns absolute value i.e. positive value.<br>SELECT ABS(-345) FROM DUAL;<br>Output: 345 |
| CEIL(data) | It returns smallest integer greater than or equal to data.<br>SELECT CEIL(456.54) FROM DUAL;<br>Output: 457 |
| FLOOR(data) | It returns largest integer less than or equal to data.<br>SELECT FLOOR(456.54) FROM DUAL;<br>Output: 456 |
| LN(data) | It returns natural logarithm of data. |
| LOG(b, data) | It returns base 'b' logarithm of data. |
| MOD(data, y) | It returns the modulus of dividing data by 'y'. |
| POWER(data, y) | It returns data raised to the power of 'y'. |
| ROUND(data, n) | It returns round value where 'n' is number of decimal place.<br>SELECT ROUND(456.54, 0), ROUND(456.54, 1) FROM DUAL;<br>Output: 457 456.5 |
| SQRT(data) | It returns square root of data.<br>SELECT SQRT(9) FROM DUAL;<br>Output: 3 |
| TRUNC(data, n) | It returns truncated value where 'n' is number of decimal places for truncation.<br>SELECT TRUNC(456.54, 0), TRUNC(456.54, 1) FROM DUAL;<br>Output: 456 456.5 |

1. Abs – Absolute Value

```
SQL> select abs(-34) from dual;

  ABS(-34)
----------
        34
```

2. cell -Returns smallest integer greater than or equal to data.

```
SQL> select CEIL(456.54) from dual;

CEIL(456.54)
--------------------
         457
```

3. floor - Returns largest integer less than or equal to data.

```
SQL> select floor(456.54) from dual;

FLOOR(456.54)
---------------------
          456
```

4. mod - Returns Modulus

```
SQL> select mod(7,2) from dual;

  MOD(7,2)
----------
         1
```

5. power –

```
SQL> select power(7,2) from dual;

POWER(7,2)
----------
        49

SQL>  select power(7,2) , power(5,3) from dual;

POWER(7,2) POWER(5,3)
---------- ----------
        49        125
```

6. round - Returns round vales where n is the number of decimal place.

```
SQL> select round(456.54 , 0) , round (456.54 , 1) from dual;

ROUND(456.54,0) ROUND(456.54,1)
--------------- ---------------
            457           456.5
```

7. Sqrt -Return Square Root of the Number

```
SQL> select sqrt(25) from dual;

 SQRT(25)
----------
         5
```

8. Trunc – Returns Truncated vale where n is the number of decimal place.
   Difference B/w Round & Trunc

```
SQL> select round(456.67 , 1) from dual;

ROUND(456.67,1)
---------------
          456.7

SQL> select trunc(456.67 , 1) from dual;

TRUNC(456.67,1)

---------------

          456.6
```

**DUAL** - The **DUAL** table is a special one-row, one-column table present by default in Oracle and other database installations. In Oracle, the table has a single VARCHAR2(1) column called DUMMY that has a value of 'X'. It is useful because Oracle doesn't allow statements like

```
SELECT 3+4
```

You can work around this restriction by writing

```
SELECT 3+4 FROM DUAL
```

## Dual Examples:

```
SQL> select sum(3+5) from dual;
SUM(3+5)
---------
```

```
         8
SQL> select CURRENT_DATE from dual;

CURRENT_D
-----------------
20-SEP-20

SQL> select sysdate from dual;

SYSDATE
---------
20-SEP-20

SQL> select ascii('A') from dual;

ASCII('A')
----------
        65

SQL> select ascii('a') from dual;

ASCII('A')
----------
        97

SQL> select next_day ('21-sep-20','monday') from dual;

NEXT_DAY(
---------------
28-SEP-20
```

Date / other Functions

```
SQL> insert into birth values('aradhna','14-mar-1999');

1 row created.
```

```
SQL> insert into birth values('aradhna','1998-jun-12');

insert into birth values('aradhna','1998-jun-12')
                         *
ERROR at line 1:  literal does not match format string
```

```
SQL> insert into birth values('aradhna',to_date
('1998-jun-12' , 'yyyy-mm-dd'));

   1  row created.
```

1. to_char

```
SQL> select name , to_char(dob , 'yy') from birth;

NAME           TO
-------------- --------
aman           96
priya          98
nikki          97
aradhna        99
aradhna        98
```

2. Nvl

To put default value at NULL position

```
SQL> select ename , comm from emp;

ENAME          COMM
---------- ----------
SMITH
ALLEN          300
WARD           500
JONES
MARTIN         1400
BLAKE
CLARK
SCOTT
KING
TURNER           0

10  rows selected.
```

```
SQL> select ename , nvl(comm , 5000) from emp;

ENAME          NVL(COMM,5000)
---------- ------------------------------------
SMITH                  5000
ALLEN                  300
WARD                   500
JONES                  5000
MARTIN                 1400
BLAKE                  5000
CLARK                  5000
SCOTT                  5000
KING                   5000
TURNER                 0
```

10 rows selected.

# Operators

- An operator manipulates individual data items and returns a result.
- The data items are called *operands* or *arguments*.
- Operators are represented by special characters or by keywords.
    - For example, the multiplication operator is represented by an asterisk (*) and the operator that tests for nulls is represented by the keywords `IS NULL`.

**Unary Operators**

A unary operator uses only one operand. A unary operator typically appears with its operand in the following format.

**Operator Operand**

**Binary Operators**

A binary operator uses two operands. A binary operator appears with its operands in the following format.

**Operand_1 Operator Operand_2**

**Set Operators**

Set operators combine sets of rows returned by queries, instead of individual data items. All set operators have equal precedence. Oracle Database Lite supports the following set operators.

- UNION
- UNION ALL
- INTERSECT
- MINUS

### Levels of Precedence of the Oracle Database

| Precedence Level | SQL Operator |
|---|---|
| 1 | Unary + - arithmetic operators, PRIOR operator |
| 2 | * / arithmetic operators |
| 3 | Binary + - arithmetic operators, \|\| character operators |
| 4 | All comparison operators |
| 5 | NOT logical operator |
| 6 | AND logical operator |
| 7 | OR logical operator |

## Character Operators

| Operator | Description | Example |
|---|---|---|
| \|\| | Concatenates character strings | SELECT 'The Name of the employee is: ' \|\| ENAME FROM EMP; |

Example:

```
SQL> SELECT 'The Name of the employee is: ' || ENAME
FROM EMP;

'THENAMEOFTHEEMPLOYEEIS:'||ENAME
-----------------------------------
The Name of the employee is: SMITH
The Name of the employee is: ALLEN
The Name of the employee is: WARD
The Name of the employee is: JONES
The Name of the employee is: MARTIN
The Name of the employee is: BLAKE
The Name of the employee is: CLARK
The Name of the employee is: SCOTT
The Name of the employee is: KING
The Name of the employee is: TURNER
The Name of the employee is: ADAMS

'THENAMEOFTHEEMPLOYEEIS:'||ENAME
-----------------------------------
The Name of the employee is: JAMES
The Name of the employee is: FORD
The Name of the employee is: MILLER

14 rows selected.
```

**Concatenating Character Strings**

Concatenating two character strings results in another character string.

```
SQL> SELECT CONCAT (CONCAT (ENAME, ' is a '),job)
FROM EMP WHERE SAL > 2000;

CONCAT(CONCAT(ENAME,'ISA'
-------------------------
JONES is a MANAGER
BLAKE is a MANAGER
CLARK is a MANAGER
SCOTT is a ANALYST
KING is a PRESIDENT
FORD is a ANALYST

6 rows selected.
```

**Arithmetic Operators**

| Operator | Description | Example |
|---|---|---|
| + (unary) | Makes operand positive | `SELECT +3 FROM DUAL;` |
| - (unary) | Negates operand | `SELECT -4 FROM DUAL;` |
| / | Division (numbers and dates) | `SELECT SAL / 10 FROM EMP;` |
| * | Multiplication | `SELECT SAL * 5 FROM EMP;` |
| + | Addition (numbers and dates) | `SELECT SAL + 200 FROM EMP;` |
| - | Subtraction (numbers and dates) | `SELECT SAL - 100 FROM EMP;` |

## 2.3 Character Operators

Character operators used in expressions to manipulate character strings are listed in Table 2-3.

### *Table 2-3 Character Operators*

| Operator | Description | Example |
|---|---|---|
| \|\| | Concatenates character strings | `SELECT 'The Name of the employee is: ' \|\| ENAME FROM EMP;` |
| | | `SELECT CONCAT (CONCAT (ENAME, ' is a '),job) FROM EMP WHERE SAL > 2000;`<br><br>`CONCAT(CONCAT(ENAME`<br>`------------------------`<br>`KING is a PRESIDENT`<br>`BLAKE is a MANAGER`<br>`CLARK is a MANAGER`<br>`JONES is a MANAGER`<br>`FORD is a ANALYST`<br>`SCOTT is a ANALYST`<br><br>`6 rows selected.` |

## Comparison Operators

Comparison operators used in conditions that compare one expression. The result of a comparison can be TRUE, FALSE, or UNKNOWN.

| Operator | Description | Example |
|---|---|---|
| = | Equality test. | SELECT ENAME "Employee" FROM EMP WHERE SAL = 1500; |
| !=, ^=, <> | Inequality test. | SELECT ENAME FROM EMP WHERE SAL ^= 5000; |
| > | Greater than test. | SELECT ENAME "Employee", JOB "Title" FROM EMP WHERE SAL > 3000; |
| < | Less than test. | SELECT * FROM PRICE WHERE MINPRICE < 30; |
| >= | Greater than or equal to test. | SELECT * FROM PRICE WHERE MINPRICE >= 20; |
| <= | Less than or equal to test. | SELECT ENAME FROM EMP WHERE SAL <= 1500; |
| IN | "Equivalent to any member of" test. Equivalent to "=ANY". | SELECT * FROM EMP WHERE ENAME IN ('SMITH', 'WARD'); |
| ANY/ SOME | Compares a value to each value in a list or returned by a query. Must be preceded by =, !=, >, <, <= or >=. Evaluates to FASLE if the query returns no rows. | SELECT * FROM DEPT WHERE LOC = SOME ('NEW YORK','DALLAS'); |
| NOT IN | Equivalent to "!=ANY". Evaluates to FALSE if any member of the set is NULL. | SELECT * FROM DEPT WHERE LOC NOT IN ('NEW YORK', 'DALLAS'); |
| ALL | Compares a value with every value in a list or returned by a query. Must be preceded by =, !=, >, <, <= or >=. Evaluates to TRUE if the query returns no rows. | SELECT * FROM emp WHERE sal >= ALL (1400, 3000); |
| [NOT] BETWEEN *x* and *y* | [Not] greater than or equal to *x* and less than or equal to *y*. | SELECT ENAME, JOB FROM EMP WHERE SAL BETWEEN 3000 AND 5000; |
| EXISTS | TRUE if a sub-query returns at least one row. | SELECT * FROM EMP WHERE EXISTS (SELECT ENAME FROM EMP WHERE MGR IS NULL); |
| *x* [NOT] LIKE *y* [ESCAPE *z*] | TRUE if *x* does [not] match the pattern *y*. Within *y*, the character "%" matches any string of zero or more characters except null. The character "_" matches any single character. Any character following ESCAPE is interpreted literally, useful when *y* contains a percent (%) or underscore (_). | SELECT * FROM EMP WHERE ENAME LIKE '%E%'; |
| IS [NOT] NULL | Tests for nulls. This is the only operator that should be used to test for nulls. | SELECT * FROM EMP WHERE COMM IS NOT NULL AND SAL > 1500; |

## Logical Operators

Logical operators which manipulate the results of conditions .

| Operator | Description | Example |
|---|---|---|
| NOT | Returns TRUE if the following condition is FALSE. Returns FALSE if it is TRUE. If it is UNKNOWN, it remains UNKNOWN. | SELECT * FROM EMP WHERE NOT (job IS NULL)<br><br>SELECT * FROM EMP WHERE NOT (sal BETWEEN 1000 AND 2000) |
| AND | Returns TRUE if both component conditions are TRUE. Returns FALSE if either is FALSE; otherwise returns UNKNOWN. | SELECT * FROM EMP WHERE job='CLERK' AND deptno=10 |
| OR | Returns TRUE if either component condition is TRUE. Returns FALSE if both are FALSE. Otherwise, returns UNKNOWN. | SELECT * FROM emp WHERE job='CLERK' OR deptno=10 |

## Set Operators

Set operators which combine the results of two queries into a single result .

| Operator | Description | Example |
|---|---|---|
| UNION | Returns all distinct rows selected by either query. | SELECT * FROM<br><br>(SELECT ENAME FROM EMP WHERE JOB = 'CLERK'<br><br>UNION<br><br>SELECT ENAME FROM EMP WHERE JOB = 'ANALYST'); |
| UNION ALL | Returns all rows selected by either query, including all duplicates. | SELECT * FROM<br><br>(SELECT SAL FROM EMP WHERE JOB = 'CLERK'<br><br>UNION<br><br>SELECT SAL FROM EMP WHERE JOB = 'ANALYST'); |
| INTERSECT and INTERSECT ALL | Returns all distinct rows selected by both queries. | SELECT * FROM orders_list1<br><br>INTERSECT<br><br>SELECT * FROM orders_list2 |
| MINUS | Returns all distinct rows selected by the first query but not the second. | SELECT * FROM (SELECT SAL FROM EMP WHERE JOB = 'PRESIDENT'<br><br>MINUS<br><br>SELECT SAL FROM EMP WHERE JOB = 'MANAGER'); |

## INPUT VARIABLES IN ORACLE SQL PLUS

**SQL> SELECT * FROM DEPT;**

| DEPTNO | DNAME | LOC | SALARY |
|---|---|---|---|

```
---------- -------------- ------------ --------- ------------------------------------
        10 TP CELL        NEW YORK        45000
        20 RESEARCH       DALLAS          50000
        30 SALES          CHICAGO         55000

SQL> ACCEPT DEPT CHAR PROMPT ' PLEASE ENTER DEPARTMENT
NAME : '

PLEASE ENTER DEPARTMENT NAME : SALES

SQL> SELECT * FROM DEPT WHERE DNAME = '&DEPT';
old   1: SELECT * FROM DEPT WHERE DNAME = '&DEPT'
new   1: SELECT * FROM DEPT WHERE DNAME = 'SALES'

   DEPTNO DNAME          LOC             SALARY
---------- -------------- ------------ ---------- ----------------------------------------
        30 SALES          CHICAGO         55000
```

**DIFINE VARIABLE**

```
SQL> DEFINE NAME = SWATI;
SQL> DEFINE;

DEFINE _DATE         = "12-OCT-20" (CHAR)
```

```
DEFINE _CONNECT_IDENTIFIER = "swati" (CHAR)
DEFINE _USER          = "SCOTT" (CHAR)
DEFINE _PRIVILEGE     = "" (CHAR)
DEFINE _SQLPLUS_RELEASE = "1002000100" (CHAR)
DEFINE _EDITOR        = "Notepad" (CHAR)
DEFINE _O_VERSION     = "Oracle Database 10g Enterprise Edition
Release 10.2.0.1.0 - Production
With the Partitioning, OLAP and Data Mining options" (CHAR)
DEFINE _O_RELEASE     = "1002000100" (CHAR)
DEFINE DEPT           = "SALES" (CHAR)
DEFINE NAME           = "SWATI" (CHAR)

SQL> DEFINE ID = 19;
SQL> DEFINE;

DEFINE _DATE          = "12-OCT-20" (CHAR)
DEFINE _CONNECT_IDENTIFIER = "swati" (CHAR)
DEFINE _USER          = "SCOTT" (CHAR)
DEFINE _PRIVILEGE     = "" (CHAR)
DEFINE _SQLPLUS_RELEASE = "1002000100" (CHAR)
DEFINE _EDITOR        = "Notepad" (CHAR)
DEFINE _O_VERSION     = "Oracle Database 10g Enterprise Edition
Release 10.2.0.1.0 - Production
With the Partitioning, OLAP and Data Mining options" (CHAR)
DEFINE _O_RELEASE     = "1002000100" (CHAR)
DEFINE DEPT           = "SALES" (CHAR)
DEFINE NAME           = "SWATI" (CHAR)
DEFINE ID             = "19" (CHAR)
```

## PL SQL (PROCEDURAL LANGUAGE –STRUCTURED QUERY LANGUAGE)

- Used By Oracle
- Extended version of SQL
- Used For Programming
- Pl/ sql  Block structured Language
    - 1st Part - Declaration
    - 2nd Part - Executable
    - 3rd Part - Exception Handling

Declare

---

Begin

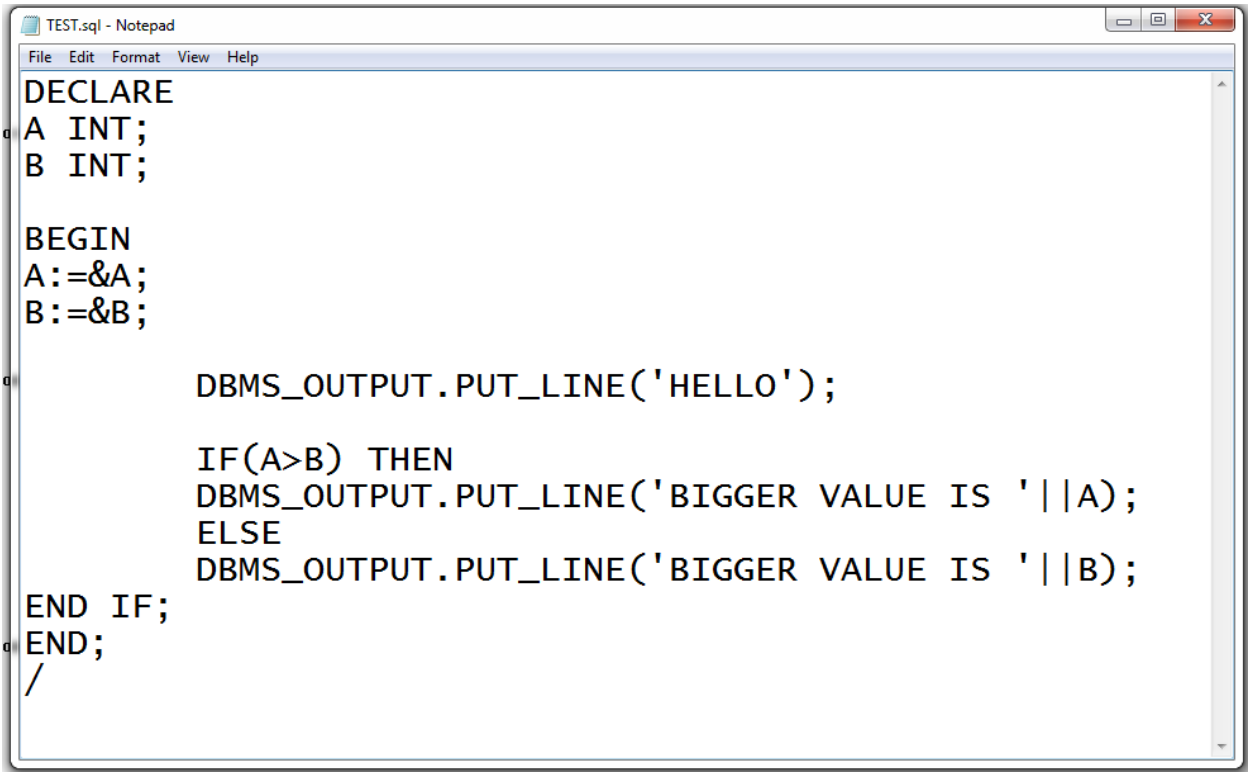
---

Exception (Optional)

---

End;
/

**SQL> SET SERVEROUTPUT ON**
**SQL> EDIT  TEST**

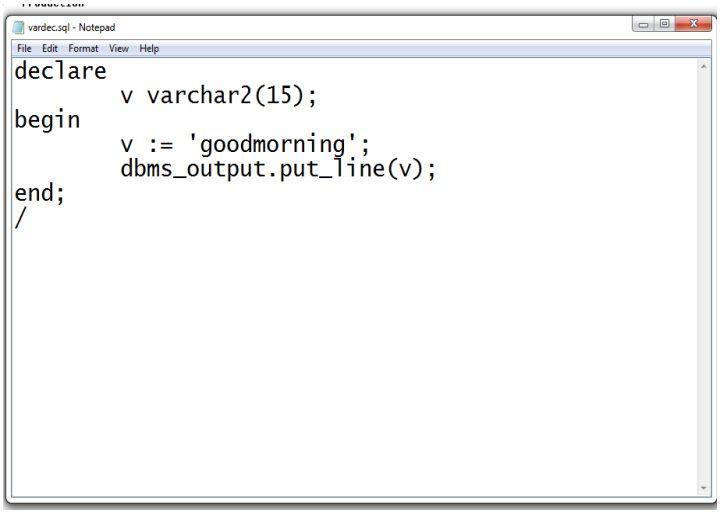



**SQL> @TEST**
**Enter value for a: 56**
**old 6: A:=&A;**
**new 6: A:=56;**
**Enter value for b: 78**
**old 7: B:=&B;**
**new 7: B:=78;**
**HELLO**
**BIGGER VALUE IS 78**

**PL/SQL procedure successfully completed.**

## To print a msg in PL/SQL

SQL> set serveroutput on
SQL> edit vardec




SQL> @vardec
goodmorning

PL/SQL procedure successfully completed.

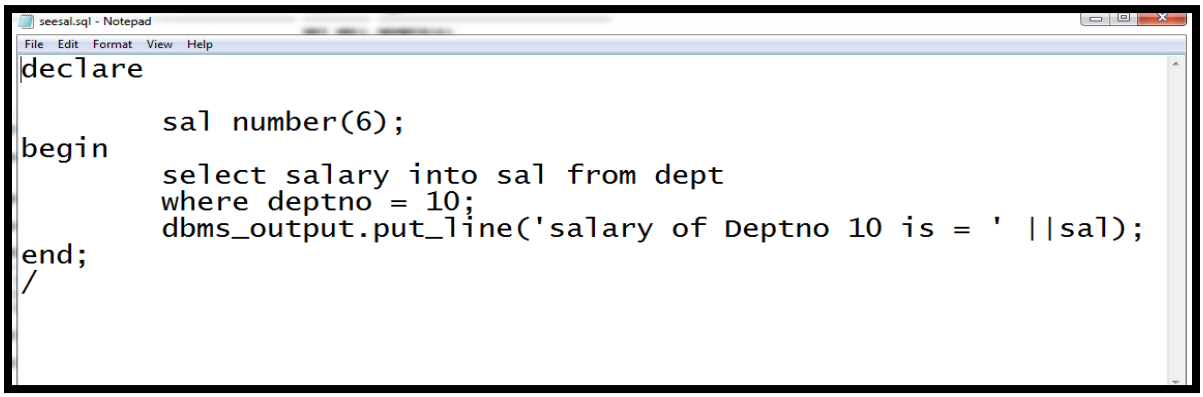
## Fetch data from Table Using Pl / Sql

SQL> select * from dept;

| DEPTNO | DNAME | LOC | SALARY |
|---|---|---|---|
| 10 | TP CELL | NEW YORK | 45000 |
| 20 | RESEARCH | DALLAS | 50000 |
| 30 | SALES | CHICAGO | 55000 |

SQL> set serveroutput on

SQL> edit seesal




SQL> @seesal

salary of Deptno 10 is = 45000

PL/SQL procedure successfully completed.

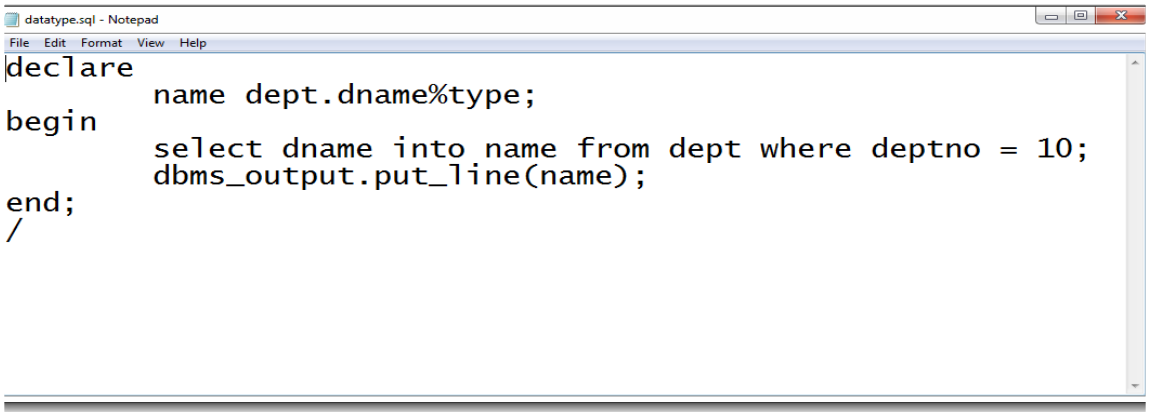
## Anchored Data Type( % type)

SQL> select * from dept;

| DEPTNO | DNAME | LOC | SALARY |
|---|---|---|---|
| 10 | TP CELL | NEW YORK | 45000 |
| 20 | RESEARCH | DALLAS | 50000 |
| 30 | SALES | CHICAGO | 55000 |

SQL> set serveroutput on

SQL> edit datatype

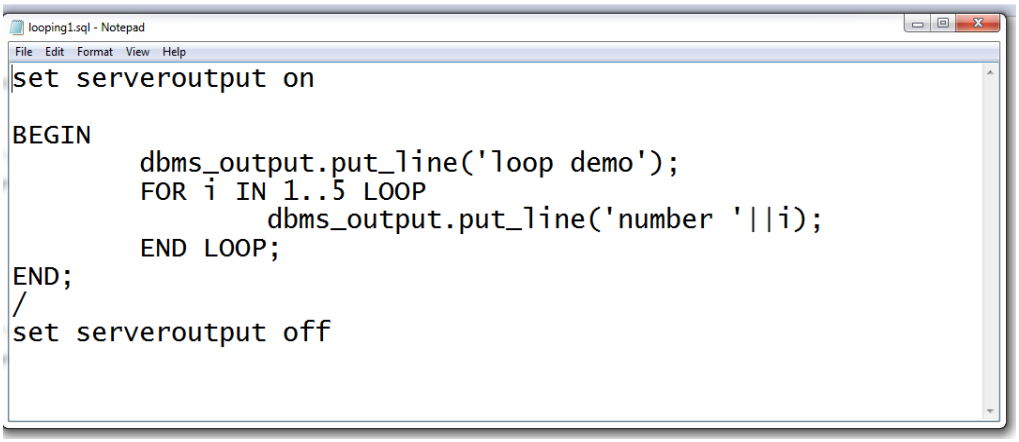



**LOOP**

- FOR
- Loop command
- WHILE

## For loop

SQL> edit looping1




SQL> @looping1

loop demo

number 1

number 2

number 3

number 4

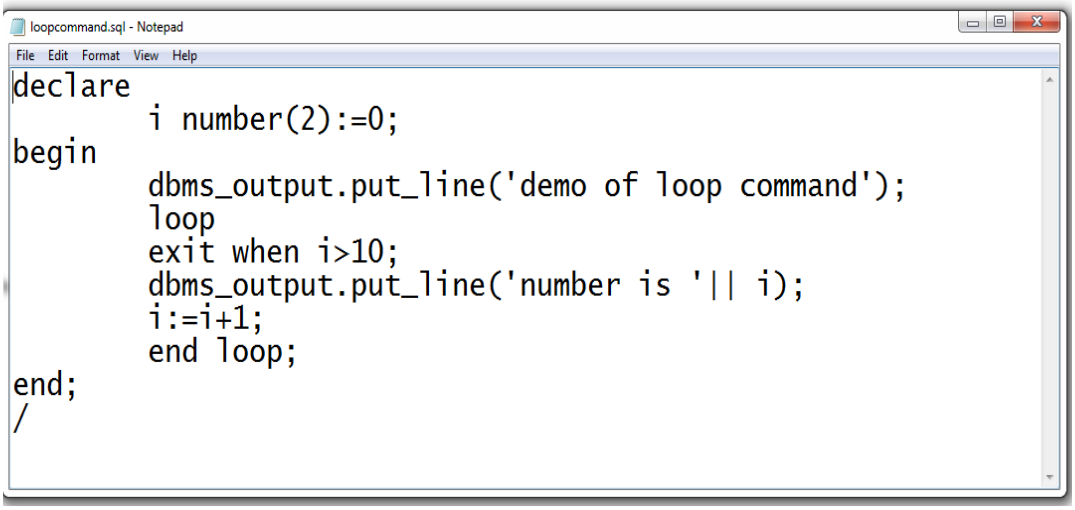
number 5

PL/SQL procedure successfully completed.

## Loop command

SQL> set serveroutput on

SQL> edit loopcommand




SQL> @loopcommand

demo of loop command

number is 0

number is 1

number is 2

number is 3

number is 4

number is 5

number is 6

number is 7

number is 8

number is 9

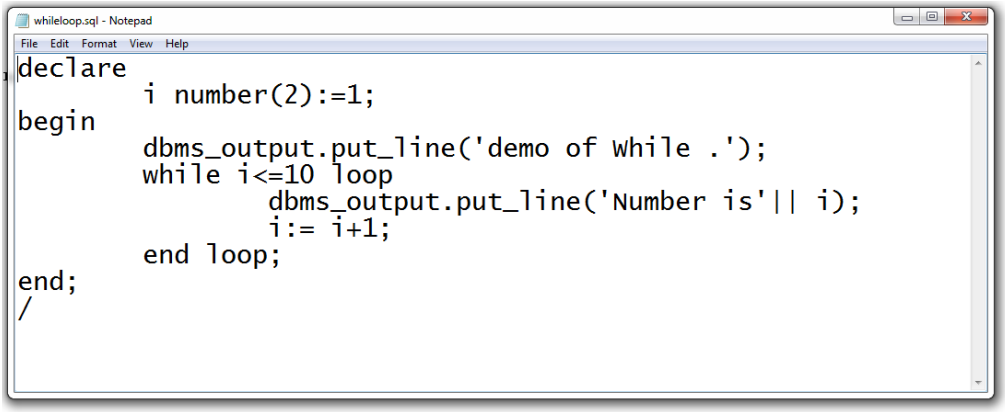
number is 10

PL/SQL procedure successfully completed.

## While loop

SQL> set serveroutput on

SQL> edit whileloop




SQL> @whileloop

demo of While .

Number is 1

Number is 2

Number is 3

Number is 4

Number is 5

Number is 6

Number is 7

Number is 8

Number is 9

Number is 10

PL/SQL procedure successfully completed.

## EXCEPTION HANDLING

1. System Defined Flag (automatically Runs)
2. User Defined Message (Overrides System Defined Flag )

**Que. Search Department Name and Salary of Department No. 10.**

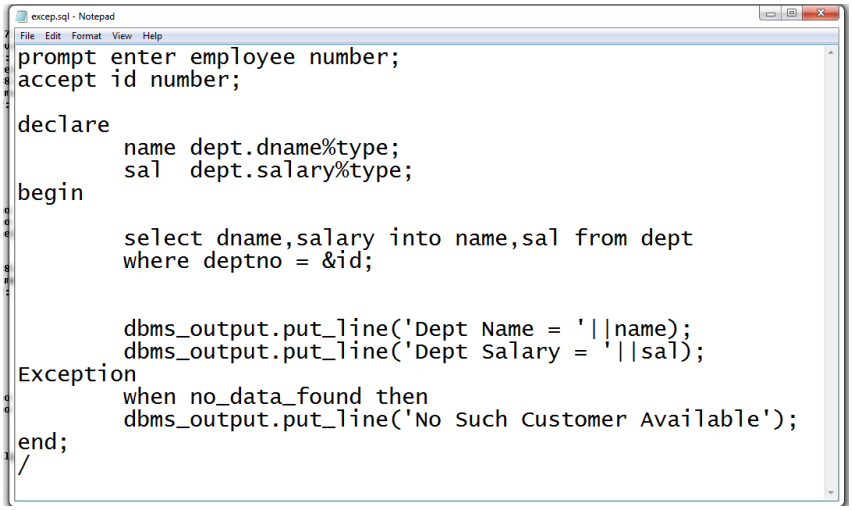
```
SQL> select * from dept;
   DEPTNO       DNAME          LOC                SALARY
---------- -------------- ------------ ---------- --------------------------------
      10          TP CELL          NEW YORK           45000
      20          RESEARCH         DALLAS             50000
      30          SALES            CHICAGO            55000
SQL> @excep
enter employee number
10
old   7:       where deptno    =       &id;
new   7:        where deptno  =        10;
Dept Name = TP CELL
Dept Salary = 45000
PL/SQL procedure successfully completed.
```

```
SQL> @excep
enter employee number
45
old   7:       where deptno = &id;
new   7:        where deptno =       45;
declare
*
ERROR at line 1:
ORA-01403: no data found
ORA-06512: at line 6
```

# Using Exception Handling

**Link** https://www.tutorialspoint.com/plsql/plsql_exceptions.htm

```
SQL> set serveroutput on
SQL> edit excep
```




```
SQL> @excep
enter employee number
10
old   7:       where deptno = &id;
new   7:       where deptno =       10;
Dept Name = TP CELL
Dept Salary = 45000

PL/SQL procedure successfully completed.
```

```
SQL> @excep
enter employee number
45
old   7:       where deptno = &id;
new   7:       where deptno =       45;
No Such Customer Available

PL/SQL procedure successfully completed.
```

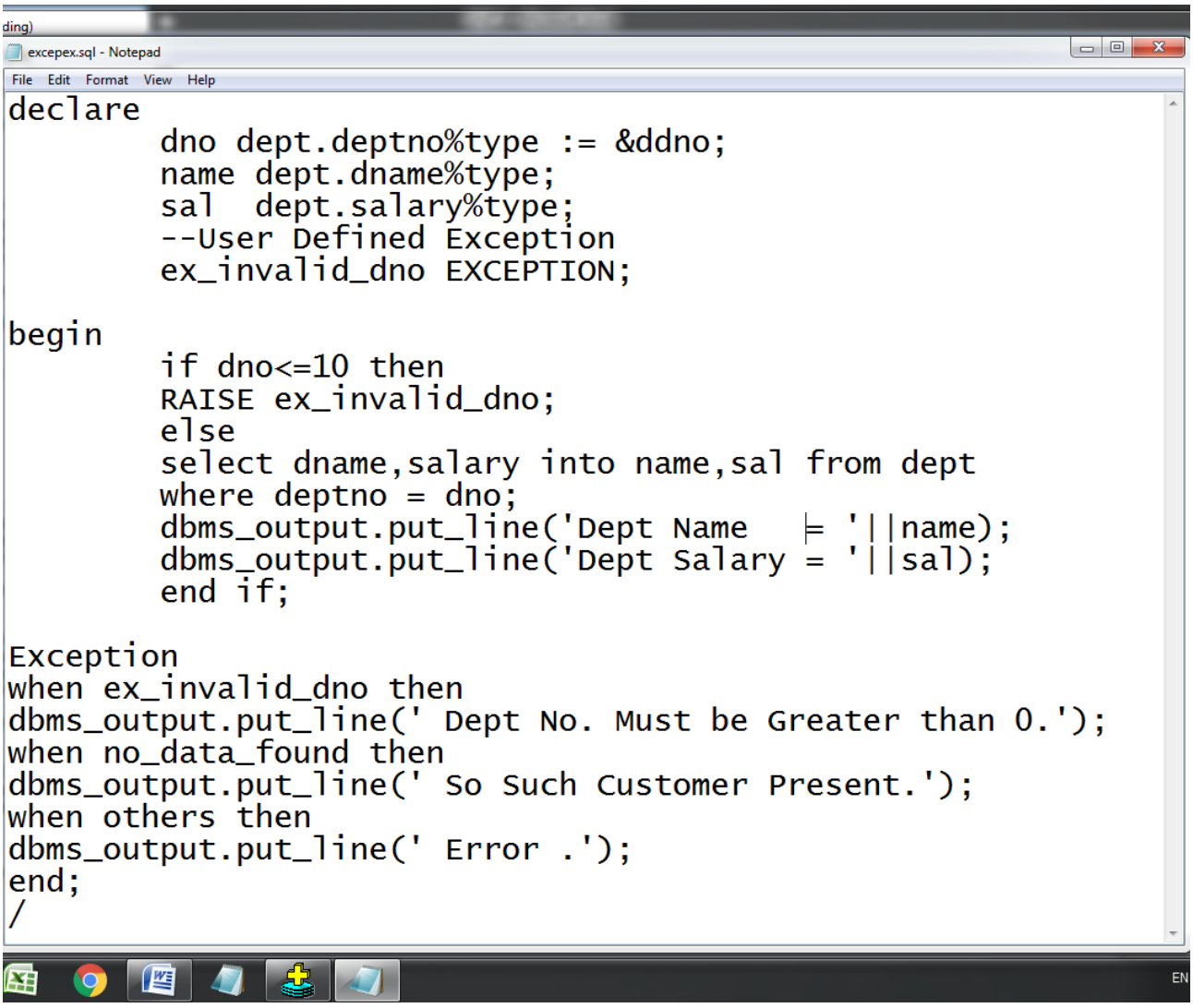
## USER DEFINED EXCEPTION HANDLING

```
SQL> select * from dept;
   DEPTNO         DNAME            LOC                        SALARY
---------- -------------- ------------- ---------- --------------------------------
```

| | | | |
|---|---|---|---|
| 10 | TP CELL | NEW YORK | 45000 |
| 20 | RESEARCH | DALLAS | 50000 |
| 30 | SALES | CHICAGO | 55000 |

SQL> set serveroutput on

SQL> edit excepex




**OUTPUT**

SQL> @excepex

Enter value for ddno: -23

```
old   2:       dno dept.deptno%type := &ddno;
new   2:       dno dept.deptno%type := -23;
Dept No. Must be Greater than 0.
PL/SQL procedure successfully completed.
```

```
SQL> /
Enter value for ddno: 45
old   2:       dno dept.deptno%type := &ddno;
new   2:       dno dept.deptno%type := 45;
So Such Customer Present.
PL/SQL procedure successfully completed.
```

```
SQL> /
Enter value for ddno: 20
old   2:       dno dept.deptno%type := &ddno;
new   2:       dno dept.deptno%type := 20;
Dept Name = RESEARCH
Dept Salary = 50000
PL/SQL procedure successfully completed.
```

**CURSOR** - To know the satatus of Last Executed DML

Oracle creates a memory area, known as the context area, for processing an SQL statement, which contains all the information needed for processing the statement; for example, the number of rows processed, etc.

A **cursor** is a pointer to this context area. PL/SQL controls the context area through a cursor. A cursor holds the rows (one or more) returned by a SQL statement. The set of rows the cursor holds is referred to as the **active set**.

You can name a cursor so that it could be referred to in a program to fetch and process the rows returned by the SQL statement, one at a time. There are two types of cursors −

1. Implicit Cursor (Declared by Oracle)
2. Explicit Cursor (Declared by User)

Implicit

- Cursor Name – Sql
- After executing of DML (Insert,Delete,Update )Command , Implicit Cursor Declared.

Implicit Cursor Attributes

- %found - Returns True / False
  - If DML is Successful Then True
  - If DML is Not Successful Then False.
- %notFound - Returns True False.
  - If DML is Unsuccessful Then True
  - If DML is Successful Then False
- %rowcount
  - Count Number of Rows.

How to use Implicit Cursor

SQL%FOUND
SQL%NOTFOUND
SQL%ROWCOUNT

**Example**

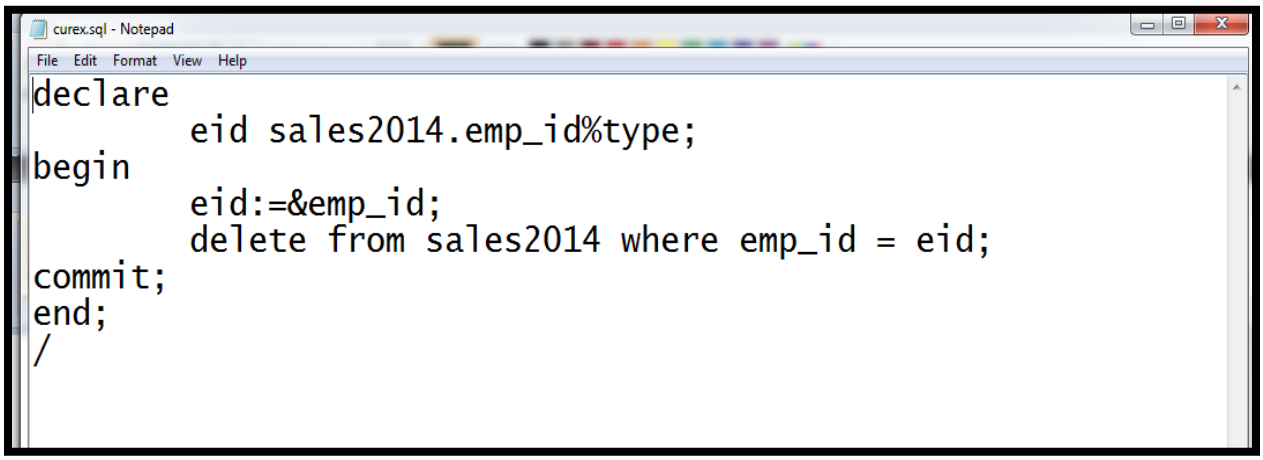
```
SQL> SELECT * FROM SALES2014;
   EMP_ID          NAME          AMOUNT
---------- ---------- ----------------------------------------
```

| | | |
|---|---|---|
| 12345 | RAM | 995.5 |
| 12455 | ROHIT | 985.59 |
| 12655 | RAGHAV | 915.59 |
| 13655 | RAGU | 905.99 |
| 13665 | RAJU | 195.99 |

```
SQL> DELETE FROM SALES2014
        WHERE EMP_ID = 11;
0 rows deleted.
```

**Using Pl/Sql without Cursor**

SQL> edit curex




```
SQL> @curex;
Enter value for emp_id: 13665
old   4:       eid:=&emp_id;
new   4:       eid:=13665;

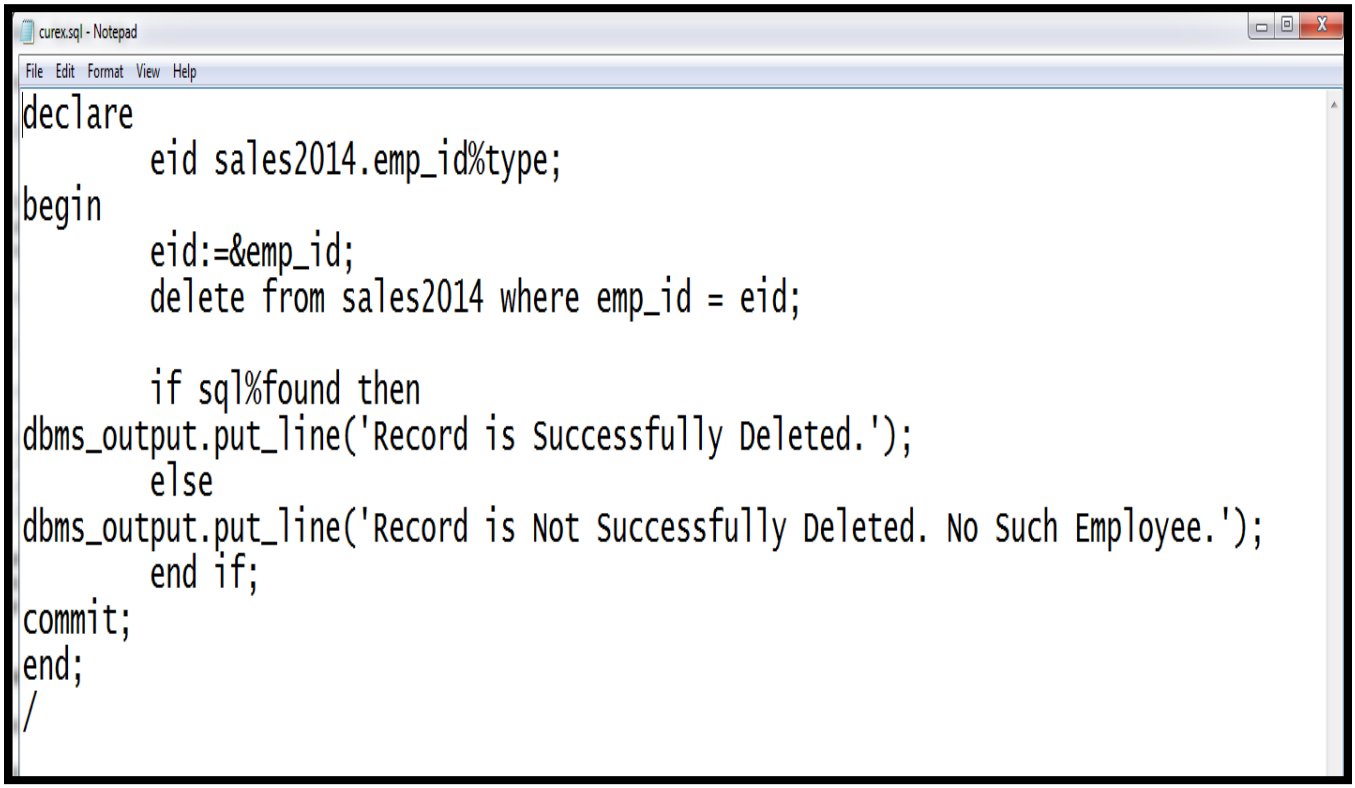
PL/SQL procedure successfully completed.
SQL> @curex;
Enter value for emp_id: 54321
old   4:       eid:=&emp_id;
new   4:       eid:=54321;
```

PL/SQL procedure successfully completed.

**How to know -whether delete is successful or Not –**

**Use Implicit Cursor**

1. If delete is successfully executed then Sql%found Returns True.
2. If delete is Not successfully executed then Sql%found Returns False.




```
SQL> select * from SALES2014;
   EMP_ID       NAME                AMOUNT
---------- ---------- ------------------------------
    12345       RAM                 995.5
```

```
   12455          ROHIT          985.59
   12655          RAGHAV         915.59
   13655          RAGU           905.99

SQL> set serveroutput on;
SQL> @curex;
Enter value for emp_id: 54321
old   4:       eid:=&emp_id;
new   4:       eid:=54321;
Record is Not Successfully Deleted. No Such Employee.
PL/SQL procedure successfully completed.

SQL> @curex;
Enter value for emp_id: 12345
old   4:       eid:=&emp_id;
new   4:       eid:=12345;
Record is Successfully Deleted.
PL/SQL procedure successfully completed.

SQL> select * from SALES2014;

   EMP_ID         NAME           AMOUNT
---------- ---------- ------------------------------
   12455          ROHIT          985.59
   12655          RAGHAV         915.59
   13655          RAGU           905.99
```

**In PL / SQL Explicit Cursor**

| In PL/SQL For | Use |
|---|---|
| | |
| 1 emp name | 1 varchar variable |
| emp name , salary | use 1 varchar variable , 1 number |

| | |
|---|---|
| | variable |
| Complete Row | Use  Row type Variable |
| All the Records (Multiple Recors) Row by Row | Use Cursor |
| | |

Declare Cursor

Open Cursor

Fetch Records from Cursor

Close Cursor

**Declare Cursor**

Cursor c1 is select * from emp;

C1 => select * from emp;

**Open Cursor**

Open <Cursor Name>;

Open c1;

**Fetch Record From Cursor**

Fetch < Cursor Name> INTO <Variable>;

Fetch c1 into x,y,z;

**Close Cursor**

Close  <Cursor Name>;

Close c1;

```
SQL> set serveroutput on;
SQL> edit excur
```

excur.sql - Notepad

File Edit Format View Help

```
declare
        i number(2):=1;
        cursor c1 is select ename, sal from emp;
        vename emp.ename%type;
        vsal   emp.sal%type;

begin
        open c1;
                loop
                        fetch c1 into vename,vsal;
                        exit when c1%notfound;
                        dbms_output.put_line(i ||'  ' ||vename ||'  '|| vsal);
                        i := i +1;
                end loop;
        close c1;
end;
/
```

**Output**

SQL> @excur;

1 SMITH 800

2 ALLEN 1600

3 WARD 1250

4 JONES 2975

5 MARTIN 1250

6 BLAKE 2850

7 CLARK 2450

8 SCOTT 3000

9 KING 5000

10 TURNER 1500

11 ADAMS 1100

12 JAMES 950

13 FORD 3000

14 MILLER 1300

PL/SQL procedure successfully completed.

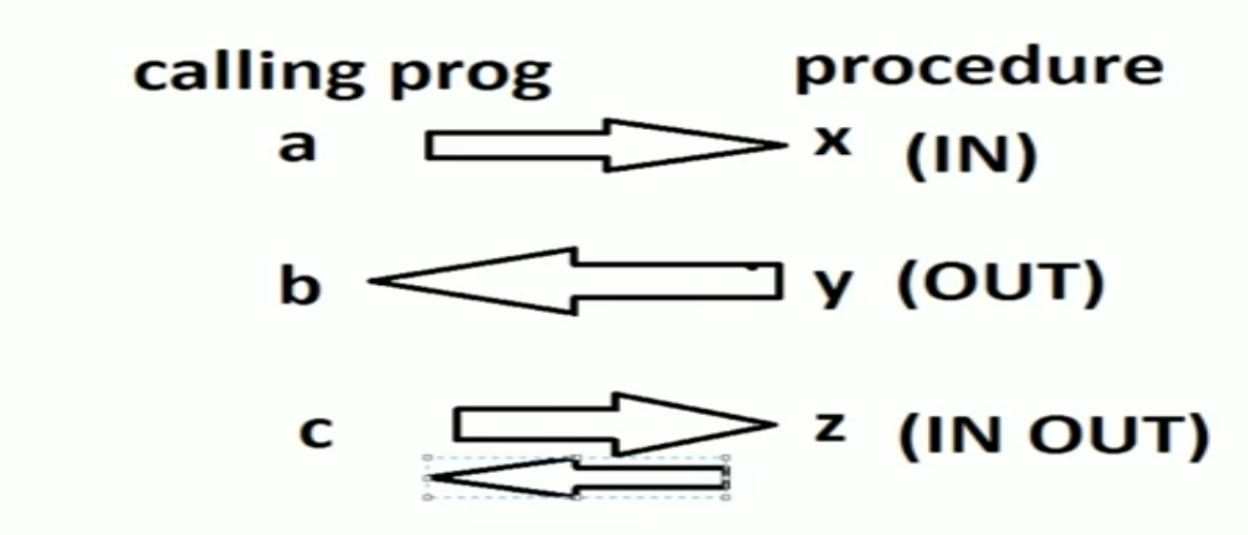
## STORED PROCEDURES

- A stored procedures is a Named Pl/Sql Block that accepts some input in the form of parameters and perform some task and may or may not returns a value.
- Procedures are created to perform one or more DML operations over Database.
- STORED PROCEDURE SYNTAX

**CREATE OR REPLACE PROCEDURE <NAME> (PARAMERTS)**
**IS**
**<VARIABLES>;**
**BEGIN**
**STATEMENTS;**
**END;;**
**END;**

PARAMETERS ARE OF 3 TYPES:

1. **IN** (BY DEFAULT) – Receives Vales
2. **OUT** – Sends Value by procedure this value is received by calling Program
3. **I**N OUT – Receives As Well As Sends Values



**Example : Increase Salary OF an Employee with EMPNO – 7369, BY amount 1000/-**

SQL> select * from SALES2014;

| EMP_ID | NAME | AMOUNT |
|---|---|---|
| 12455 | ROHIT | 985.59 |
| 12655 | RAGHAV | 915.59 |
| 13655 | RAGU | 905.99 |

```
CREATE OR REPLACE PROCEDURE
    RAISE_SALARY(E IN NUMBER,AMT IN NUMBER,S OUT NUMBER)
IS
BEGIN
  UPDATE EMP SET SAL=SAL+AMT
        WHERE EMPNO = E;
  COMMIT;
  SELECT SAL INTO S FROM EMP WHERE EMPNO=E;
END;
 /
```

To Execute created procedure from SQL Prompt

SQL> select empno,sal from emp;

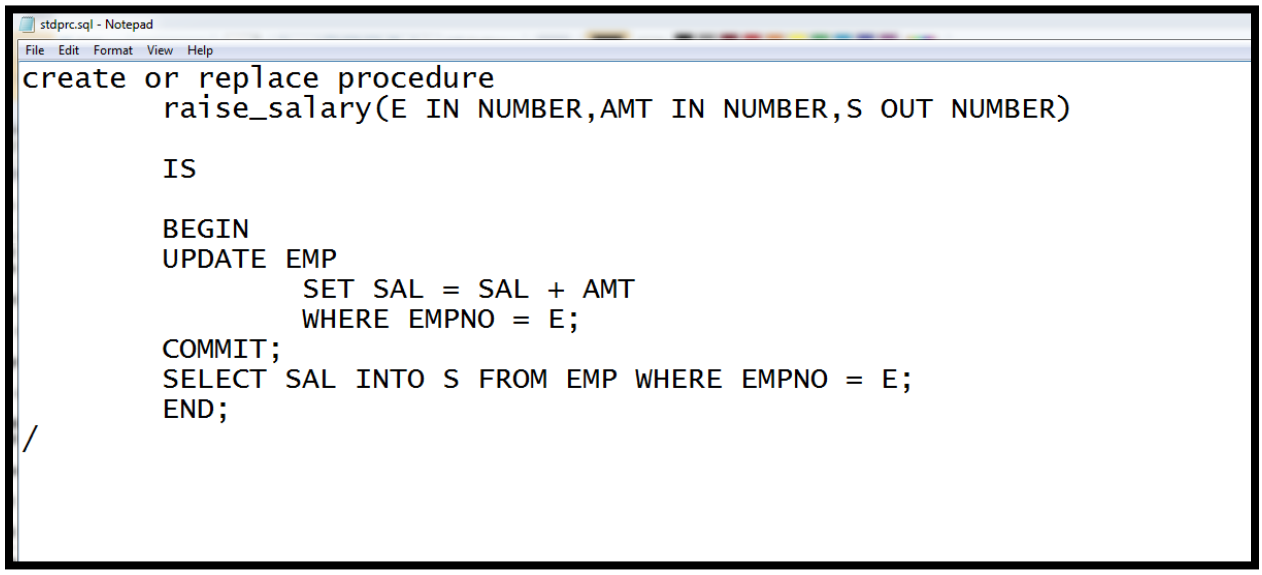
| EMPNO | SAL |
|---|---|
| 7369 | 800 |
| 7499 | 1600 |
| 7521 | 1250 |
| 7566 | 2975 |
| 7654 | 1250 |
| 7698 | 2850 |

```
      7782       2450
      7788       3000
      7839       5000
      7844       1500
      7876       1100

     EMPNO       SAL
---------- ----------
      7900        950
      7902       3000
      7934       1300

14 rows selected.
```

SQL> edit stdprc



```
SQL> @stdprc
Procedure created.
SQL> VARIABLE K NUMBER
SQL> EXECUTE RAISE_SALARY(7369,1000, :K);
PL/SQL procedure successfully completed.

S is passed to the K
SQL> print :k;

         K
----------
      1800
SQL> select empno,sal from emp;

     EMPNO       SAL
---------- ----------
      7369       1800
      7499       1600
      7521       1250
      7566       2975
      7654       1250
      7698       2850
      7782       2450
      7788       3000
      7839       5000
      7844       1500
      7876       1100

     EMPNO       SAL
---------- ----------
```

| | |
|---|---|
| 7900 | 950 |
| 7902 | 3000 |
| 7934 | 1300 |

14 rows selected.

## TRIGGER

- A trigger is a pl/sql block structure which is automatically fired when an event occurs in the database.
- The Event can be DDL , DML or System Events.
- A trigger code is automatically fired when an associated event happens in the Database.

Types:

1. DML (Insert / Update / Delete)
2. DDL ( Alter / Drop / Truncate / Grant / Rename / Revoke / Audit)
3. System ( Logon / Logoff / Startup / Shutdown)
4. Instead of Trigger ( Tigger on Views)
5. Compound ( Combines DML trigger in Single Block)

Purpose:

1. Auditing
2. Enforcing Complex Referential Integrity
3. Logging
4. Enforcing Security of Transaction
5. Data Replicating to Multiple Tables
6. Prevent Invalid Data or Transactions
7. DDL Trigger for Auditing & Logging of DDL Statements
8. DDL Trigger to Block Unwanted DDL Statements
9. System Triggers are Mainly for Auditing & Logging.
10. System Triggers to do some initial clean up activity or to initiate job based on Logon / Logoff or startup / Down.